

RESEARCH

Open Access



Balancing fitness and precision in process model repair: framework formalization, assessment, and benefits for simulation

Francesco Vinci^{1*} and Massimiliano de Leoni¹

*Correspondence:
fvinci@math.unipd.it

¹ Department of Mathematics,
University of Padua, Via Trieste,
63, Padova 35121, Italy

Abstract

Process models are essential tools for Process Science, guiding stakeholders on process automation. They are also employed to engage stakeholders into discussion to analyze how the process currently is, with the goal of discussing compliance and performance issues. They are also used as backbone to create process simulation models, with the goal of analyzing the “as-is” processes and of proposing alternative process models that guarantee improved performances. In all these settings, desirable models need to only allow all the legitimate behavior (high precision), while disallowing forbidden behavior (high fitness). Often, models fail to achieve these properties and need to be repaired. This paper proposes a model-repair framework that assembles process-simulation techniques, Machine-learning models, and Explainable AI, aiming to pinpoint the parts of the process models that enable illegitimate behavior and/or disallow valid one. These indications are then used to automatically repair a model and achieve better balances fitness and precision. Our framework has been implemented and evaluated on several models for five processes. The experimental results show that we outperform the state of the art in terms of balancing fitness and precision. The practical relevance of this technique is further showcased by illustrating the benefits of our repairing process models as backbone to create more accurate simulation models, capable to simulate more realistic runs. The simulation of more realistic process runs ensures that any process simulation-driven process improvements lead to advancements that exist in the real process world and do not remain the simulated world.

Keywords: Process model repair, Business process simulation, Machine learning, Explainable AI

Introduction

A process model is a foundational tool in process science, offering a comprehensive representation of the sequences of activity executions that the process allows. Process models are used to engage process stakeholders into discussions on how processes should be executed, or are alternatively used as input for Process-aware Information Systems to automate processes. A process model is also the backbone of business process simulation models (cf. van der Aalst (2015)), which are basically process models extended

with the run-time characterization for simulation (inter-arrival time, activity durations, branching probabilities, etc.).

Desirable models need to be precise and only allow legitimate behavior, while they should be able to represent the valid executions that have been observed (high fitness). As an example, an imprecise process model would lead to a business simulation model that enables runs that would not be observed in reality, while a low-fitness model would ultimately cause some legitimate to never be simulated: in both cases, process analysts would gain insight on the basis of unrealistic simulations.

When process models may fail to achieve satisfactory level of fitness and precision, they need to be repaired. However, fitness and precision are often contrasting: improving one may worsen the other.

This paper reports on a novel control-flow model-repair framework that tries to improve the harmonic means of fitness and precision, so as to provide a better balancing between these two dimensions. The framework can be summarized by the schema illustrated in Fig. 1. The key idea is to leverage on Business Process Simulation: the original process model, namely before any repair, is used to generate a set of simulated traces. These traces are then compared with those of a real event log to highlight the differences. This comparison is based on training a Machine Learning (ML) model that, given an execution trace, is capable to classify whether it is from the real or simulated event log. The rationale is that, if the process model is characterized by high fitness and precision, the ML model is not capable to determine whether a trace belongs to the real or simulated event log. In other words, the lower is the harmonic means of fitness and precision in a process model, the higher is the accuracy of the ML model. When the ML model accuracy is high, it is possible to pinpoint which trace characteristics are used for the determination whether the trace is from the real or simulated event log. In this paper, we use the SHAP (SHapley Additive exPlanations) method to pinpoint the differences. These trace characteristics are then employed to automatically improve the process model, aiming to achieve a process model that, once simulated, would produce traces that are indistinguishable from the real event log.

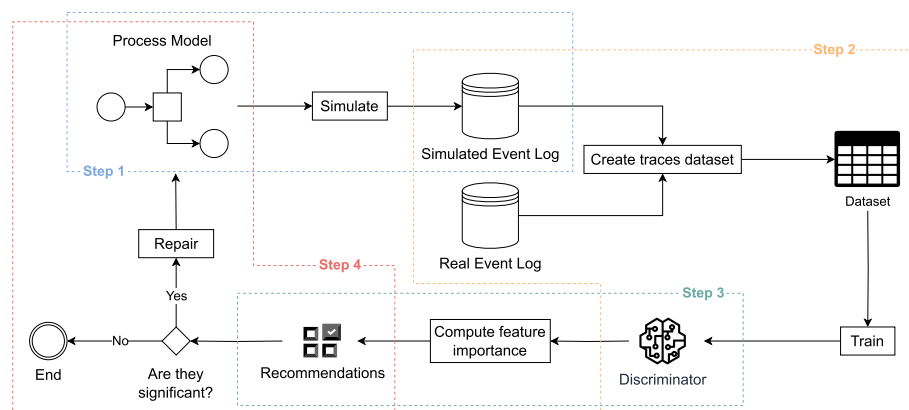


Fig. 1 The schema of the proposed framework. Step 1 provides a simulated event log; in Step 2 a dataset of real and simulated traces is created for training a Machine Learning discriminator model; Step 3 regards the computation of the feature importance, providing recommendations for repairing the model; in Step 4 the recommendations are used for automating process model repairs

The framework includes a *Repair* step (cf. Fig. 1), where approaches need to be operationalized to use the recommendations for repair. In Vinci and de Leoni (2024), we proposed two different approaches for repairing the model, hereafter referred to as **basic** and **greedy**. However, these approaches assume that the same activity cannot be repeated in different parts of the model. In this article, we propose a third, alternative approach that lifts this assumption, which is based on log-model alignments.

To demonstrate the effectiveness of our proposed technique, we conducted evaluations on five distinct real-life processes and various models, which were literature hand-made or mined models. Two of the literature hand-made models are characterized by same activities being present in different parts of the model, which could only be improved via the new approach that, leveraging on log-model alignments, allow the same activity to appear at different model's parts

The results showcase the technique's ability to enhance the original process models, improving on both fitness and precision. The evaluation also compares the resulting repaired models with respect to those obtained by Fahland and van der Aalst (2015), again highlighting how we achieve a better balance between fitness and precision.

This article also reports on the benefits of better balancing fitness and precision when one aims at business process simulation: the repaired model is being used as basis to create a simulation model. The evaluation of the same case studies illustrate that the resulting simulation model is more realistic: this is testified by the fact that the simulation model produces a set of process execution, in fact a simulated event log, which is closer to the original, real-life event log, compared with the simulated models built upon the original model and upon the model repaired by the technique by Fahland and van der Aalst (2015).

The rest of the paper is organized as follows. “[Preliminaries](#)” section reports on preliminary concepts, “[Framework](#)” section introduces the framework, “[Evaluation](#)” section outlines the evaluation and results. “[Towards multi-perspective process model repair](#)” section discusses how our model repair technique, which is currently focused on the control-flow perspective, can be extended to incorporate the other perspectives, namely data, resource and time: in particular, a proposal towards multi-perspective process repair is proposed, along with delineating open challenges. “[Related works](#)” section reports the related works, and finally “[Conclusion](#)” section concludes the paper.

Preliminaries

This section introduces the preliminary concepts useful to discuss the framework. First, basic definitions of events, traces, and event logs are presented, subsequently, a subsection recalls the notions on Petri nets. A final subsection provides the definitions related to the explainability of a Machine Learning model.

Events, traces, event logs

Usually process transactional data are collected as event logs. An event log is a collection of traces, where a trace is a sequence of events with their attributes. In this paper, we abstract the events as the activities that are executed, thus ignoring the other event attributes because they are irrelevant for our technique.

Definition 1 (Traces & Event Logs). Let $\mathcal{A}$ be the set of activity labels. A trace σ is a sequence $\langle \text{start}, e_1, \dots, e_m, \text{end} \rangle$, where $\langle e_1, \dots, e_m \rangle \in \mathcal{A}^*$, where start and end are special start-of-trace and end-of-trace delimiter. An event log $\mathcal{L}_{\mathcal{A}}$ is a multiset of traces, i.e. $\mathcal{L}_{\mathcal{A}} \in \mathbb{B}(\mathcal{A}^*)$ ¹.

The repair technique defined hereafter does not necessarily require to have the start-of- and end-of-trace delimiter. This choice allows a lighter formalization of the technique, when defining the set of trace features.

In the following we define a relation function that will be used to encode the traces for the training of the discriminator model.

Definition 2 (Directly-Follow Relation). Given a trace $\sigma = \langle e_0, \dots, e_m \rangle \in \mathcal{L}_{\mathcal{A}}$, a pair $(a, b) \in \mathcal{A} \times \mathcal{A}$ belongs to the footprint relation $>_{\sigma}$ iff $\exists i \in \{0, \dots, m-1\}$ s.t. $e_i = a$ and $e_{i+1} = b$.

In the remainder, we use the notation $a >_{\sigma} b$ to indicate $(a, b) \in >_{\sigma}$.

Petri nets

In this paper we use Petri nets for modeling and representing processes with a set of recorded instances $\mathcal{L}_{\mathcal{A}}$. We opted for Petri nets because they have a simple yet formal syntax and semantics, sufficiently reach to model the aspects relevant for our framework.

Definition 3 (Petri Net). A Petri net is a triple (P, T, F) , where P is a finite set of places, T is a finite set of transitions, and $F \subseteq (P \times T) \cup (T \times P)$ is a set of arcs.

In the remainder, for any $t \in T$, $t^{\bullet} = \{p \mid (p, t) \in F\}$ and ${}^{\bullet}t = \{p \mid (p, t) \in F\}$ are the set of places with arcs coming from or going to t , respectively. The state of a Petri net is identified by its current marking:

Definition 4 (Marking). A marking m of a Petri net (P, T, F) is a function $m : P \rightarrow \mathbb{N}$ which assigns for each place $p \in P$ the number $m(p)$ of tokens at this place. A Petri net system is a Petri net with an initial marking m^I and a final marking m^F .

A **Labelled Petri net system** $N = (P, T, F, \lambda, m^I, m^F)$ consists of a Petri net (P, T, F) , a labelling function $\lambda : T \rightarrow \mathcal{A} \cup \{\tau\}$ where $\mathcal{A}$ is a set of activities and τ denotes the label of invisible transitions, m^I the initial marking and m^F the final marking. Hereafter, we use the term Petri net to indicate a Labelled Petri net system, without causing ambiguities.

Since Petri nets are used to model business processes, we assume that they are in fact Workflow Nets (van der Aalst 1998): there exist an input place $i \in P$ and an output place $o \in P$ that have no incoming or outgoing arcs, respectively, as well as every node is in a path between i and o .

The behaviour of a Petri net can be represented as a so-called **reachability graph**, which is a way of representing the states of Petri nets. A reachability graph of a Petri net N is a directed graph $G_N = (V, E)$, where:

¹ $\mathbb{B}(\mathcal{A}^*)$ indicates the set of all the multisets with element in $\mathcal{A}^*$.

1. V is the set of nodes where each node $v \in V$ represents a reachable marking (from the initial m^I);
2. E is the set of edges, each of which is a triple (m_i, t, m_j) , which indicates that t can fire at marking m_i , leading to marking m_j .

In the remainder, given two activities a and b , $a >_N b$ is used to indicate that, according to N , activity b can be immediately executed after a , namely E contains a set of edges $\{(m_1, t_1, m_2), \dots, (m_{n-1}, t_{n-1}, m_n)\}$ such that $\lambda(t_1) = a$, $\lambda(t_{n-1}) = b$ and, for all $1 < k < n - 1$, $\lambda(t_k) = \tau$.

This paper aims to obtain sound models. Since we focus on Petri net, we base on the **soundness** criteria introduced in van der Aalst (1998), according to which a Petri net $(P, T, F)S$ is sound iff:

- *Option to complete*: For any reachable marking m , it is possible to reach marking m^F .
- *Proper completion*: The only reachable marking that contains a token in the final place o is the marking m^F .
- *Absence of dead transitions*: There are no dead transitions, i.e. it does not exist a transition that is not enabled at any reachable marking.

Log-model alignments

Event logs are sequences of events, each associated to an activity label, while replaying a Petri nets generates firing sequences of transitions. Our repairing technique requires one to unambiguously map events with their activity labels to Petri-net transition firings. The Petri-net labelling function λ , which associates activity labels to transitions, allows for two transitions to be associated with the same activity labels. In this case, the mapping is far from being trivial, because the exact association of an event to a transition in case of multiple transitions with the same label require to look at the entire trace as a whole, knowing that some transitions may not have a label, and the corresponding events do not leave a trail in the event log.

In literature, this problem can be coped with by constructing an alignment of event log and process model: events in the event log need to be related to model elements and vice versa. An alignment relates *moves in log* and to *moves in model* as explained in the following definition. Here, we explicitly indicate *no move* with $\gg$.

Definition 5 (Alignment). Let $N = (P, T, F, \lambda, m^I, m^F)$ Labelled Petri net system with $\lambda : T \rightarrow \mathcal{A} \cup \{\tau\}$. Let us denote $S_L = \mathcal{A} \cup \{\gg\}$ and $S_M = T \cup \{\gg\}$. A pair $(s_L, s_M) \in (S_L \times S_M) \setminus \{(\gg, \gg)\}$ is

- a *move in log* if $s_L \in \mathcal{A}$ and $s_M = \gg$,
- a *move in model* if $s_L = \gg$ and $s_M \neq \gg$,
- a *synchronous move* if $s_L \neq \gg$, $s_M \neq \gg$, and $\lambda(s_M) = s_L$.

$\Sigma = (S_L \times S_M) \setminus \{(\gg, \gg)\}$ is the set of the *legal moves*.

The *alignment* of a log trace $\sigma \in \mathcal{A}^*$ and a Petri net N is a sequence $\gamma \in \Sigma^*$ such that the projection on the first element (ignoring $\gg$) yields σ , and the projection on the second element yields a valid firing sequence of transitions of N .

An alignment of the event log $\mathcal{L}$ and a Petri net N is a multiset $\mathcal{S} \in \mathcal{B}(\Sigma^*)$ of alignments such that, for each log trace $\sigma \in \mathcal{L}$, there exists an alignment $\gamma \in \mathcal{S}$ of σ and N . $\mathcal{S}$ is a multi-set because an event log may contain the same log trace multiple times, potentially resulting in multiple identical alignments.

In order to quantify the severity of a deviation, a cost function $\kappa \in (\Sigma \rightarrow \mathbb{R}_0^+)$ needs to be defined over the legal moves. This cost function can be generally customized, as a deviation severity depends on the process' domain. With a cost function at hand, the cost of an alignment γ is defined as the sum of the costs of the individual moves in the alignment, i.e., $\mathcal{K}(\gamma) = \sum_{(s_L, s_M) \in \gamma} \kappa(s_L, s_M)$.

It may be defined differently for individual processes, since, generally speaking, the costs depend on the specific characteristics of the process. In most scenarios, a **standard cost function** is applicable: $\forall (s_L, s_M) \in \Sigma. \kappa^{std}(s_L, s_M) = 1$ if $s_L = \gg$ or $s_M = \gg$; otherwise, $\kappa^{std}(s_L, s_M) = 0$. The cost of an alignment γ is defined as the sum of the costs of the individual moves in the alignment, i.e., $\mathcal{K}(\gamma) = \sum_{(s_L, s_M) \in \gamma} \kappa(s_L, s_M)$.

In the remainder, we use the *standard cost function* that assigns cost 1 to each log move and to each model for a transition t with an associated label (i.e., s.t. $\lambda(t) \neq \tau$), and gives cost 0 to any other legal move.

In general, the number of potential alignments is infinite, but we are not interested in any, but in an alignment with the lowest cost, also known as **optimal alignment**. Note that there may be multiple alignments that are optimal: without further information for discriminating, any of alternative optimal alignment can be considered. van der Aalst et al. (2012) proposes techniques to compute optimal alignments.

The repair approach in “[Repair via log-model alignments](#)” section, which is based on log-model alignments, requires to map each alignment prefix γ' with the corresponding marking reached after firing the transition sequence in the process projection of γ' . These alignments with associated markings are hereafter named **extended alignment**:

Definition 6 (Extended Alignment). Let $\gamma = \langle (s_L^1, s_M^1), \dots, (s_L^n, s_M^n) \rangle$ be the alignment of a Labelled Petri net system $N = (P, T, F, \lambda, m^I, m^F)$ with a trace σ . The extended alignment γ^{ext} of γ is a sequence of triple $\langle (s_L^1, s_M^1, m_1), \dots, (s_L^n, s_M^n, m_n) \rangle$ where, for all $1 \leq j \leq n$, m_j is the marking that is reached from m^I after firing the sequence of transitions $\langle s_M^1, \dots, s_M^j \rangle$, ignoring the instances of $\gg$.

In fact, to keep the explanation simpler, we use the term alignment to refer to the extended alignment, avoiding any ambiguities.

Explanations of ML models

In Machine and Deep Learning, the highest accuracy is often achieved by using complex models that are difficult to interpret and, therefore, are often referred to as black box models. In a general sense, a classifier or regressor can be abstracted as a function ψ that returns the estimated value of the target variable, defined over a domain Y . The ψ 's

domain is defined over the Cartesian product over the domains $X_1, \dots, X_n$ of the independent features $f_1, \dots, f_n$, namely $\psi : X_1 \times \dots \times X_n \rightarrow Y$.

Several approaches have been introduced to address the problem of the accuracy-interpretability trade-off. One of the most widespread is based on computing Shapley values, which can be efficiently computed via the method SHAP (SHapley Additive exPlanations) (Lundberg and Lee 2017). Shapley values are computed for each feature of each potential element $\mathbf{x} = (x_1, \dots, x_n) \in X_1 \times \dots \times X_n$: given a feature f_i , the Shapley value $\phi_i^{\mathbf{x}}$ intuitively expresses how much the feature value x_i contributes to the model prediction when the input is $\mathbf{x}$. The contribution is computed as the average prediction difference with respect to a certain base value, when feature f_i takes on value x_i and each other feature is either discarded or retained. The base value is computed as the average of the predictions when a given multiset $\mathcal{X} \in \mathbb{B}(X_1 \times \dots \times X_m)$ is used as input (usually the model's training set).

Shapley values are computed for individual input vectors and individual features. However, we need to obtain a general contribution value for each individual feature, fairly independent of the specific input. We thus introduce the following explanation function, used thereafter in the paper, that returns the average of the Shapley values when a feature assumes a certain value.

Definition 7 (Explanation function). Let $\mathcal{F} = \{f_1, \dots, f_m\}$ be the set of all the features, with X_j be the domain of f_j . Let $\psi : X_1 \times \dots \times X_m \rightarrow Y$ be a regressor or a classifier model. Let $\mathcal{X} \in \mathbb{B}(X_1 \times \dots \times X_m)$ be a multiset of elements in $X_1 \times \dots \times X_m$. Let us consider any $f_i \in \mathcal{F}$. Given a value $x \in X_i$, we define

$$\phi_{f_i}(x, \mathcal{X}) = \underset{\substack{\mathbf{x} = (x_1, \dots, x_m) \in \mathcal{X} \\ x_i = x}}{\text{avg}} (\phi_i^{\mathbf{x}}) \quad (1)$$

as the average of the Shapley values $\phi_i^{\mathbf{x}}$ computed over the set of input values $\mathbf{x} \in \mathcal{X}$, when the feature f_i assumes values $x \in X_i$.

Intuitively, $\phi_{f_i}(x, \mathcal{X})$ expresses the expected impact of feature f_i taking on value x , i.e. how much the fact $f_i = x$ influences the prediction.

Framework

The overall idea of this paper is to run simulations over a process model, to obtain a simulated event log and then compare it to the real one using a Machine Learning discriminator model. Then, SHAP is used to compute the discriminating features, pinpoint the differences between the two event logs, and repair the control-flow process model in an automated iterative way.

The framework can be divided into four steps (see Fig. 1). In Step 1, the initial process model is used to generate a simulated event log. In Step 2, the traces of the real and simulated event logs are encoded yielding to a dataset used by a ML discriminator model to detect the inaccuracies of the process model. Step 3 focuses on the identification of the discriminating features, measuring the SHAP feature importance, which is then used in

a Step 4 to repair the process model. If the discriminations are significant, the four steps are repeated; otherwise, the repairing procedure terminates.

“Step 1: process simulation” to “Step 4: process model repair” sections describe the details of the four steps of the framework. Particularly, “Step 4: process model repair” section introduces three kinds of possible approaches for repairing the process model. Hereafter, we refer to the first as the **basic approach** (cf. “The basic approach” section). The approach outlined in “Greedy repair” section will be named the **greedy approach**, and the one presented in “Repair via log-model alignments” section will be referred as the **log-model alignments approach**.

Step 1: process simulation

The framework takes in input an event log $\mathcal{L}_{\mathcal{A}}^r$ over an activity set $\mathcal{A}$, along with a Petri net $N = (P, T, F, \lambda, m^I, m^F)$ with $\lambda : T \rightarrow \mathcal{A} \cup \{\tau\}$ (cf. “Petri nets” section).

The event log is temporally split into training and validation sets: the first part of the traces is allocated to the training set and the remaining traces to the validation set. The former is used to train the model and discover the discriminatory features, while the latter to validate the tuning of hyper-parameters, aiming to avoid underfitting and overfitting of the discriminator model.

Petri nets are converted into Stochastic Labelled Petri nets (Burke et al. 2021), which assign a firing probability to each transition. In particular, we compute these probabilities as discussed in Burke et al. (2021), on the basis of the frequencies of transition firings.

The simulation is executed for a number of process instances equal to the number of traces in the original event log $\mathcal{L}_{\mathcal{A}}^r$. A simulated trace is in fact obtained by looking at the occurrence sequence of the fired, labelled transitions. The simulated event log $\mathcal{L}_{\mathcal{A}}^s$ is then divided into a train and a validation set, with the same number of traces of the splitting in the real event log. This balanced distribution facilitates a fair comparison and evaluation of the framework performance against real data.

Example 1 Let $\mathcal{L}_{\mathcal{A}}^r$ be an event log:

$$\mathcal{L}_{\mathcal{A}}^r = \{ \langle a, b, d, i, f, g \rangle, \langle a, e, b, c, d, f, g \rangle, \langle a, b, c, d, e, f, g \rangle, \langle a, b, c, d, e, i, f, h \rangle \}$$

and N the Petri net illustrated in Fig. 2. Each transition of the Petri net is assigned a weight based on the frequency of its occurrence in the optimal alignments. Then, given a marking, the probability of firing a certain transition t is computed as the ratio between the weight of t and the sum of the weights of all enabled transitions at that marking.

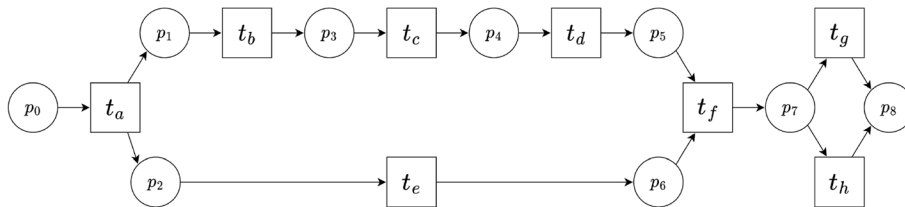


Fig. 2 Petri net example. Circles and squares represents respectively places and transitions. We indicate visible transition with label / with t_i

For example, the probability of firing the transition labelled with g , when it is enabled, is 0.75. The Stochastic Petri net can then be used for obtaining a simulated event log:

$$\mathcal{L}_{\mathcal{A}}^s = \{ \langle a, b, c, e, d, f, g \rangle, \langle a, b, c, d, e, f, g \rangle, \langle a, b, e, c, d, f, g \rangle, \langle a, e, b, c, d, f, h \rangle \}$$

Step 2: detection of inaccuracies of simulation models

In the second phase, the real and simulated event logs, $\mathcal{L}_{\mathcal{A}}^r$ and $\mathcal{L}_{\mathcal{A}}^s$, are used to population the directly-follow feature set $\mathcal{F}_{\mathcal{A}}^> = \{f_{a>b} \mid (a, b) \in \mathcal{A} \times \mathcal{A}\}$. Note that, since we define a trace to always have the activities $\text{start}, \text{end} \in \mathcal{A}$, features $f_{\text{start}>a}$ and $f_{a>\text{end}}$ belong to $\mathcal{F}_{\mathcal{A}}^>$, with $a \in \mathcal{A} \setminus \{\text{start}, \text{end}\}$.

The creation and population of these features are in accordance to the following mapping function:

Definition 8 (Trace-to-Feature Mapping Function). Let $\mathcal{L}_{\mathcal{A}}$ be an event log over a set of activity labels $\mathcal{A}$. Let $\mathcal{F}_{\mathcal{A}}^>$ be the set of directly-follow features. We define a trace-to-feature mapping function $\rho^{\mathcal{L}_{\mathcal{A}}} : \mathcal{L}_{\mathcal{A}} \rightarrow (\mathcal{F}_{\mathcal{A}}^> \rightarrow \{0, 1\})$ s.t. $\rho^{\mathcal{L}_{\mathcal{A}}}(\sigma) = v^{\sigma}$ where, given $\sigma \in \mathcal{L}_{\mathcal{A}}$, $\forall f_{a>b} \in \mathcal{F}_{\mathcal{A}}^>$:

$$v^{\sigma}(f_{a>b}) = \begin{cases} 1 & \text{if } a >_{\sigma} b \\ 0 & \text{otherwise} \end{cases} \quad (2)$$

With these concepts at hand, the training dataset $T_{\mathcal{L}_{\mathcal{A}}^r, \mathcal{L}_{\mathcal{A}}^s}$ of the discriminator model is constructed as follows²:

$$T_{\mathcal{L}_{\mathcal{A}}^r, \mathcal{L}_{\mathcal{A}}^s} = \bigsqcup_{\sigma \in \mathcal{L}_{\mathcal{A}}^r} (\rho^{\mathcal{L}_{\mathcal{A}}}(\sigma), 1) \uplus \bigsqcup_{\sigma \in \mathcal{L}_{\mathcal{A}}^s} (\rho^{\mathcal{L}_{\mathcal{A}}}(\sigma), 0) \quad (3)$$

where 1 and 0 indicate that the trace belongs to the real and simulated event logs respectively.

Example 2 Given the real event log

$$\mathcal{L}_{\mathcal{A}}^r = \{ \langle a, b, d, i, f, g \rangle, \langle a, e, b, c, d, f, g \rangle, \langle a, b, c, d, e, f, g \rangle, \langle a, b, c, d, e, i, f, h \rangle \}$$

and the simulated one

$$\mathcal{L}_{\mathcal{A}}^s = \{ \langle a, b, c, e, d, f, g \rangle, \langle a, b, c, d, e, f, g \rangle, \langle a, b, e, c, d, f, g \rangle, \langle a, e, b, c, d, f, h \rangle \}$$

Table 1 describes the training dataset: each row represents the encoding of a trace, where the last column indicates if the trace belongs to the real or simulated event log (1 if real, 0 otherwise), and the others the correspondent features. For example, considering the trace $\sigma = \langle a, b, d, i, f, g \rangle$, the value of the features $f_{a>b}$, $f_{b>d}$, $f_{d>i}$, $f_{i>f}$ and $f_{f>g}$ are 1 since $a >_{\sigma} b$, $b >_{\sigma} d$, $d >_{\sigma} i$, $i >_{\sigma} f$ and $f >_{\sigma} g$ while the other features take value 0.

² Symbol $\uplus$ indicates the union of multisets.

Table 1 Training dataset example: each row represents the encoding of a trace, *target* is equal to 1 if the trace is real, 0 if simulated. For space reasons, we only show some features. The others follow the same reasoning

	$f_{a>b}$	$f_{a>e}$	$f_{b>c}$	$f_{b>d}$	...	$f_{e>f}$	$f_{e>i}$	$f_{i>f}$	<i>target</i>
$\langle a, b, d, i, f, g \rangle$	1	0	0	1	...	0	0	1	1
$\langle a, e, b, c, d, f, g \rangle$	0	1	1	0	...	0	0	0	1
$\langle a, b, c, d, e, f, g \rangle$	1	0	1	0	...	1	0	0	1
$\langle a, b, c, d, e, i, f, h \rangle$	1	0	1	0	...	0	1	1	1
$\langle a, b, c, e, d, f, g \rangle$	1	0	1	0	...	0	0	0	0
$\langle a, b, c, d, e, f, g \rangle$	1	0	1	0	...	1	0	0	0
$\langle a, b, e, c, d, f, g \rangle$	1	0	0	0	...	0	0	0	0
$\langle a, e, b, c, d, f, h \rangle$	0	1	1	0	...	0	0	0	0

The constructed dataset may have variables with a high correlation, leading to collinearity. When collinearity exists, it can be a challenge to interpret the individual effects of these variables on the target variable. In fact, it becomes difficult to discern whether a feature has a significant impact on the target variable independently or whether its effect is captured by other correlated features in the model. To overcome these issues, a feature selection is performed, considering the correlation between the features: the framework iterates across the features, and for each one, it considers the set of the features with an absolute correlation greater than a given threshold, only retaining one for each of these correlation groups, whose choice can be arbitrarily done³. Furthermore, the features that are associated with a variance lower than a fixed threshold are removed. The validation set is then constructed following the same procedure and using the same features selected by the feature selection phase described above.

Then a ML model is developed to predict if traces are real or simulated: the output of the model has to be intended as the probability that the input trace is real. The model is trained using the training set $T_{\mathcal{L}^r_{\mathcal{A}}, \mathcal{L}^s_{\mathcal{A}}}$, defined in Eq. 3, and the validation set is used to validate the procedure and fine-tune the hyper-parameters of the model.

Step 3: identification of the discriminating features

Once the model is trained, we create a multiset $\mathcal{X}^{true}$ that is obtained from the training-set instances that the ML model has correctly classified, projected over the independent features (i.e., removing the target variable)⁴. This is done in order to understand the set of features that enable a correct discrimination. We compute the impact of the features in the output for the samples in $\mathcal{X}^{true}$ to provide explanations of certain classifications (cf. “Explanations of ML models” section). For each activity pair $a, b \in \mathcal{A}$:

- If the feature $f_{a>b}$ taking value 1 (or 0) increases (decreases, resp.) the probability for a trace to belong to the real event log, i.e. $\phi_{f_{a>b}}(1, \mathcal{X}^{true}) > 0$ ($\phi_{f_{a>b}}(0, \mathcal{X}^{true}) < 0$,

³ Usually, suitable thresholds are above 0.9. In our evaluation, we set it to 0.99.

⁴ The ML model returns a probability value between 0 and 1. Therefore, an instance with a true value of 1 is considered as correctly classified if the probability value is larger than 0.5. Similarly, an instance with a true value of 0 is correctly classified if the value is smaller than 0.5.

resp.), it means that the model does not sufficiently allow for a being followed by b . Therefore, the model needs to enable this.

- If $\phi_{f_{a>b}}(1, \mathcal{X}^{true}) < 0$ (or $\phi_{f_{a>b}}(0, \mathcal{X}^{true}) > 0$), the feature $f_{a>b}$ taking value 1 (0, resp.) impacts the prediction to classify a trace as belonging to the simulated (real, resp.) event log. This means that the real event log does not show that behavior.

In the remainder, $\mathcal{R}$ and $\overline{\mathcal{R}}$ are respectively the set of features related to behavior that needs to be included or not to the model, namely the first or second case in the list above. At this stage, the features that were removed because they were strongly correlated are considered again: any feature exhibiting a strongly positive correlation with another within the set $\mathcal{R}$ is incorporated into $\mathcal{R}$. Similarly, if any feature is negatively correlated to any other in $\overline{\mathcal{R}}$, then it is added to $\mathcal{R}$.

Example 3 Let us assume that a Machine Learning model M has been trained on the basis of the data set in Table 1, which encodes the traces in both real and simulated event logs. When the same dataset, excluding the *target* variable, is used for testing, a value between 0 and 1 is predicted, as shown in the column *prediction value* in Table 2: values below 0.5 are going to be treated as if the *predicted* target variable takes on value 0 (trace classified as belonging to the *real* trace), whereas values above 0.5 are going to be treated as if it takes on value 1 (*simulated* trace).

In Table 1, one can observe that the predicted value of the second and third trace - highlighted in red - are far below 0.5, thus predicting these traces belong to the real event log: however, those traces are wrongly predicted, because they belong to the simulated event logs. After removing the vectors for those two traces and filtering out the *target* feature/column, the resulting set forms the support set $\mathcal{X}^{true}$ required by SHAP (cf. “[Explanations of ML models](#)” section). Now, the Shapley values can be computed. Suppose to have $\phi_{f_{b>d}}(0, \mathcal{X}^{true}) = -0.25$. This means that when the feature $f_{b>d}$ takes value 0 in a trace, it causes a decrease of 0.25 in the model’s prediction output, thereby reducing the probability to be classified as *real* (and increasing the probability of it being classified as *simulated*). This means that traces in which activity b is not followed by d are more likely being part of original event log than part of the simulated

Table 2 Predictions of the Machine Learning model example. Column *actual target value* represent if the trace is real or simulated (1 and 0 respectively), while column *predicted value* the output of the Machine Learning model. The output has to be intended as the probability of the trace to be real. Green values represent correct predictions (i.e. if the value is closest to the target value), red values wrong predictions

	$f_{a>b}$	...	$f_{i>f}$	<i>actual target value</i>	<i>predicted value</i>
$\langle a, b, d, i, f, g \rangle$	1	...	1	1	0.99
$\langle a, e, b, c, d, f, g \rangle$	0	...	0	1	0.10
$\langle a, b, c, d, e, f, g \rangle$	1	...	0	1	0.45
$\langle a, b, c, d, e, i, f, h \rangle$	1	...	1	1	0.90
$\langle a, b, c, e, d, f, g \rangle$	1	...	0	0	0.15
$\langle a, b, c, d, e, f, g \rangle$	1	...	0	0	0.45
$\langle a, b, e, c, d, f, g \rangle$	1	...	0	0	0.10
$\langle a, e, b, c, d, f, h \rangle$	0	...	0	0	0.10

event log, which indicates that the original model should be repaired so as to allow activity d to follow b . Similar reasoning can be done because $\phi_{f_{i>f}}(1, \mathcal{X}^{true}) = 0.2$, meaning that when the feature $f_{i>f}$ takes value 1 in a trace, it causes the increasing of 0.2 in the prediction output: the possibility for activity f to follow i should be incorporated into the repaired model.

Step 4: process model repair

In this final step, the set of features $\mathcal{R}$ is used for repairing the Petri net $N = (P, T, F, \lambda, m^I, m^F)$, with $\lambda : T \rightarrow \mathcal{A} \cup \{\tau\}$, where $\mathcal{A}$ is a set of activity labels, and $G_N = (V, E)$ is the reachability graph. We assume the Petri net N to be sound and safe (van der Aalst 1998). This is not very limiting: several process-discovery algorithms, such as Inductive Miner (Leemans et al. 2013) and Split Miner (Augusto et al. 2017), produce Petri nets for which safety holds; also BPMN models with the typical constructs (AND and XOR splits and joins) can be modeled as a safe Petri net. Assuming soundness is also very reasonable: unsound process models cannot be used to enact processes, making them impractical and essentially futile. Repairing models to ensure soundness is orthogonal to what proposed here, and may be a preliminary step before applying our framework.

The remainder of this section proposes three different approaches to repair the process model to incorporate the direct-following relation $a > b$ for each discriminating feature $f_{a>b} \in \mathcal{R}$. “[The basic approach](#)” section reports on a first basic approach, which requires each visible transition to be associated to a different label, namely two different visible transitions can be associated to the same label. The basic approach repairs the Petri net by introducing several new transitions, which can be slow to perform, on the one hand, and can potentially hamper the net’s precision and simplicity, on the other hand. Greedy approach introduced in “[Greedy repair](#)” section implements some heuristics that enables introducing a smaller subset of the changes, if compared with the basic approach, although this might negatively affect the quality of the repairing model. Last and very importantly, “[Repair via log-model alignments](#)” section proposes an alternative approach based on log-model alignments, which allows multiple visible transitions to be associated to the same activity labels.

The basic approach

As mentioned, the basic technique requires each visible transition to be associated to a different label. In “[The basic approach](#)” section, we indicate with t_l the transition with label $l \in \mathcal{A}$, i.e. $\lambda(t_l) = l$.

For any feature $f_{a>b} \in \mathcal{R}$ we should consider four categories: (i) when there exist net transitions with labels a and b in the original Petri net N , (ii) and (iii) when one of them is missing, and (iv) when both are missing. The model is repaired differently for each category:

- (i) If $a \not\prec_N b$ and $t_a, t_b \in T$, we alter the Petri net such that the corresponding reachability graph contains one edge (m'_i, τ_{ij}, m''_j) , for each marking $m'_1, \dots, m'_n \in V$ reached after firing transition t_a and for each marking $m''_1, \dots, m''_q \in V$ in which t_b is enabled. This is pictorially represented in Fig. 3a. The transitions τ_{ij} with

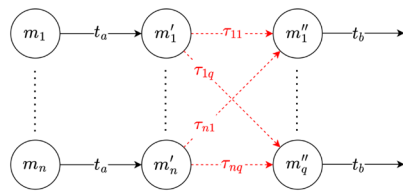
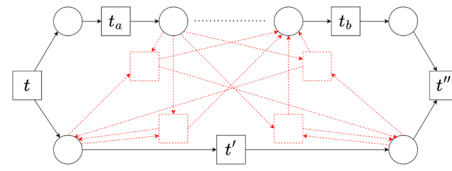
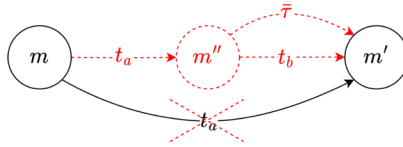
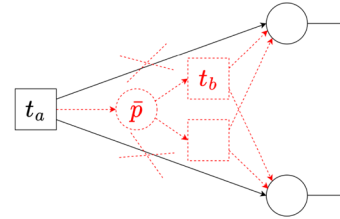
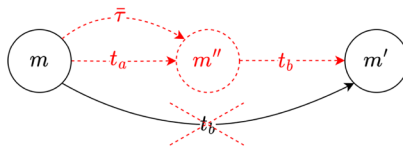
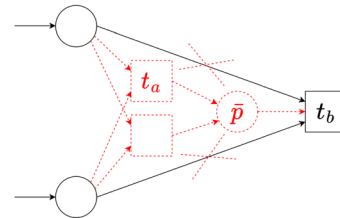
(a) Reachability graph: $t_a, t_b \in T$.(b) Petri net: $t_a, t_b \in T$.(c) Reachability graph: $t_a \in T, t_b \notin T$.(d) Petri net: $t_a \in T, t_b \notin T$.(e) Reachability graph: $t_a \notin T, t_b \in T$.(f) Petri net: $t_a \notin T, t_b \in T$.

Fig. 3 Petri net repairs. Red and dashed parts denote repairs. We indicate visible transition with label / with t_l , i.e. $\lambda^{-1}(l) = t_l$ if $l \neq \tau$. We assume, without loss of generality, that in category (i) (Fig. 3b), there is only one parallelism involving a single transition. For categories (ii) and (iii) (Fig. 3d–3f), we consider places to be disjoint. Any differences are negligible

$1 \leq i \leq n$ and $1 \leq j \leq q$ are invisible (i.e. $\lambda(\tau_{ij})$ is set to τ) and added to the Petri net. Figure 3b illustrates the resulting Petri net, in a case in which $n = 2$ and $q = 2$, where the inserted invisible transitions are shown in red. The above-mentioned changes in the reachability graph correspond to these specific changes in the Petri net because the Petri net is safe.

(ii) If $t_b \notin T$, we alter the Petri net as follows:

- for each $p \in t^\bullet$, we remove the arc (t_a, p) from F ;
- we introduce a visible transition t_b with $\lambda(t_b)$ set to b , an invisible $\bar{\tau}$, and a place $\bar{p}$;
- for each $p \in t^\bullet$, we add the arcs (t_b, p) and $(\bar{\tau}, p)$, along with the arcs $(t_a, \bar{p})$, $(\bar{p}, t_b)$, $(\bar{p}, \bar{\tau})$.

These changes are visually depicted in Fig. 3d, which refers to a case where the outgoing places from t_a are two. Note the category (ii) is directly elaborated with respect to the Petri net, because the discussion is simpler. Of course, these changes in the Petri net imply modifications in the reachability graph, as illustrated in Fig. 3c.

- (iii) If $t_a \notin T$ the procedure is similar to the category (ii) where now we consider the places in $\bullet t_b$. Figure 3f and e ought to be sufficient for a full understandability.
- (iv) If $t_a, t_b \notin T$, then no repair is carried out. However, if $f_{a>b}$ is a discriminatory feature, there is surely a feature $f_{d>a}$ (or $f_{b>d}$, resp.) for some activity d^5 . The model would then be repaired wrt. $f_{d>a}$ (or $f_{b>d}$, resp.), namely the category (ii) or (iii) would be applied. This would incorporate transition t_a (or t_b) in the model, bringing this repair to category (ii) or (iii), which will be incorporated through the subsequent framework iteration.

The proposed repairs guarantee the soundness and safeness of the repaired Petri net if the original one is sound and safe:

Theorem 1 *Let $N = (P, T, F)$ be a sound and safe Petri net system, then the Petri net system N' obtained after applying all repair changes related to every category, namely (i), (ii) or (iii), is sound and safe.*

Proof The safeness of N' can be easily proven: the only new markings are m'_i in categories (ii) and (iii), then it is safe by construction.

Regarding soundness, it is sufficient to prove the soundness properties (cf. “Petri nets” section) for every category.

Option to Complete. For category (i), the reachability graph of the repaired Petri net does not include new states/markings and no arcs are removed: only new arcs are inserted. Therefore, if every marking m of the original reachability graph allowed reaching the final marking m^F , then the same marking m still allows m^F to be reached, at least using the same path. For categories (ii) and (iii), the reachability graph is changed similarly to what depicted in Fig. 3c and e, respectively. From the newly-introduced marking m'' , it is possible to reach marking m' , which allowed reaching the final marking in the original Petri net, and so like in the repaired Petri net. Similarly, the original marking m now enables arriving at marking m'' , from which, as just mentioned, it is possible to reach the final marking: therefore, it is possible to reach the final marking from m .

Proper completion. Let us define $o \in P$ as the final place, namely $m^F(o) = 1$ and, $m^F(p) = 0$ for all $p \in P \setminus \{o\}$. For category (i), this is straightforward, as no new markings are included and the initial Petri net is sound. For category (ii), we need to prove that $m''(o) = 0$. Let us assume $m''(o) > 0$ by contradiction. Since the repaired Petri net is safe, there must be $m''(o) = 1$. There are two cases: $m' \neq m^F$ and $m' = m^F$. If $m' \neq m^F$, $m'(o) = 0$ (cf. Fig. 3c). Since $m''(o) = 1$, when firing, transitions t_b and $\bar{\tau}$ consume the token in o . However, by construction (cf. Fig. 3d), these transitions only consume one token from $\bar{p}$: then, $m''(o) = 0$, contradicting the hypothesis. If $m' = m^F$, $m'(o) = 1$. Since, by hypothesis, $m''(o) = 1$ and transitions t_b and $\bar{\tau}$, when firing, produce one token in o , hence $m'(o) = 2$, which contradicts the safeness properties. Category (iii) is analogous.

Absence of dead transitions. Every transition is present in the reachability graph of the repaired Petri net, which implies that no transition is dead. $\square$

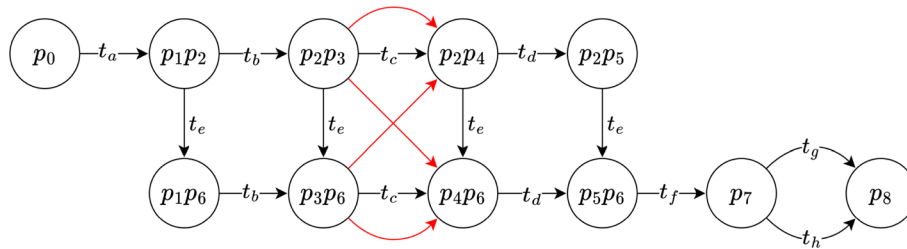
⁵ It might unlikely be the case that d is not present, either. However, this would not invalidate the reasoning, because one can reiterate it until one transition is found.

Note that, since the repaired Petri net is kept sound and safe after each change, the different changes can be applied in order. Also, after applying all changes, the entire four phases of our framework can be reiterated. Indeed, during each iteration, the Machine Learning model may uncover new discriminating features, as the training set adapts in response to the evolving process model.

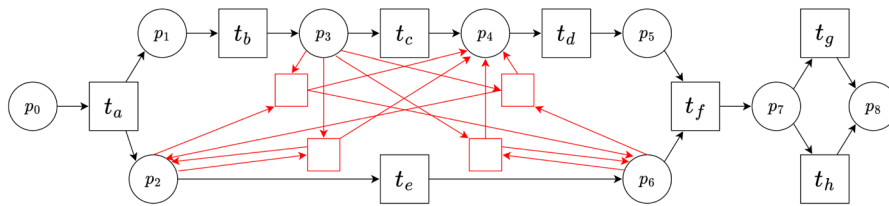
Example 4 Consider the Petri net N illustrated in Fig. 2. To include the relation $b >_N d$, the reachability graph is updated, as depicted in Fig. 4a, resulting in the Petri net shown in Fig. 4b. If we also want to incorporate the relation $i >_N f$, the final Petri net is as shown in Fig. 4c.

Greedy repair

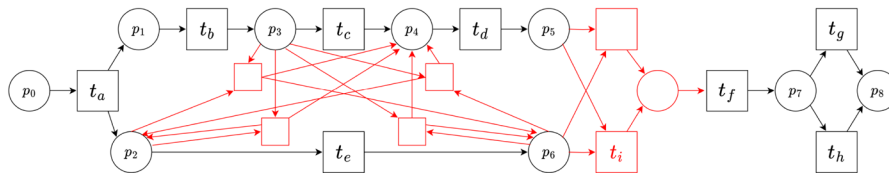
Category (i) introduced in “The basic approach” section triggers the introduction of several arcs between markings in the reachability graph $G_N = (V, E)$, and consequently brings the introduction of multiple invisible transitions in the original Petri net



(a) Reachability graph after including $b >_N d$.



(b) The repaired Petri net for including $b >_N d$.



(c) The repaired Petri net for including $i >_N f$.

Fig. 4 Petri net repairs example. Red parts denote repairs. Markings in the reachability graph are indicated as multisets. We indicate visible transitions with label l with t_l , i.e. $\lambda^{-1}(l) = t_l$ if $l \neq \tau$

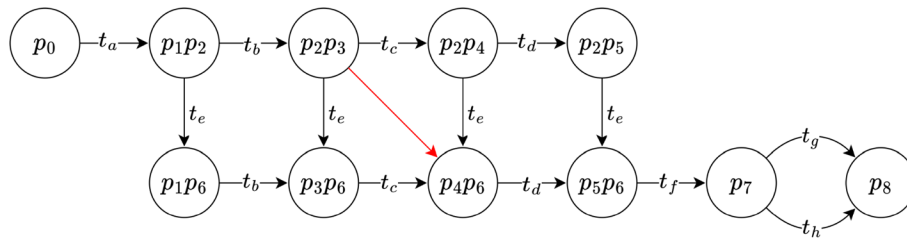
$N = (P, T, F, \lambda, m^I, m^F)$. The allowance of new behavior may negatively affect the resulting Petri net models, due to the reduction of model simplicity.

To tackle this challenge, we propose a greedy approach to include relation $a >_N b$ between two transitions $t_a, t_b \in T$. This greedy approach comes from the observation that, given any marking $m_a \in G_N$ reached after firing t_a and any marking $m_b \in G_N$ where t_b is enabled, it is sufficient to add arc (m_a, τ, m_b) to formally incorporate $a >_N b$ into the Petri net. In particular, the approach picks the marking m_a with the shortest path in G_N from the initial marking m^I , along with marking m_b with the short path to the final marking m^F . The approach is motivated by the fact that, by selecting such these two nodes, fewer paths are included in the reachability graph: this leads to a minimization of the behaviour additionally included into the Petri net model, thereby reducing the negative impacts on the precision.

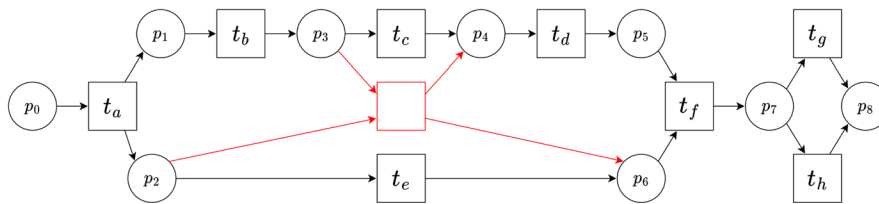
Example 5 Given the Petri net N in Fig. 2, if we want to introduce the relation $b >_N d$ using the greedy approach, only one invisible transition is added to the Petri net model which corresponds to the new arc in the reachability graph highlighted in red in Fig. 5a.

Of course, we acknowledge that this approach only introduces a subset of the potential changes needed to maximize fitness, and, even if we want to introduce a single invisible transition, the choice does not guarantee to be the best to repair the model. This explains why the approach is indeed labelled as greedy.

Moreover, the greedy repair approach also leads to a significant reduction in computational costs compared to the basic repair method. Specifically, the basic repair sustains a computational cost of $\mathcal{O}(|E| + |V|^2)$. This includes $\mathcal{O}(|E|)$ for identifying the nodes to include in the repair by searching across the arcs, and $\mathcal{O}(|V|^2)$ for adding the



(a) The reachability graph after greedy repair.



(b) The repaired Petri net.

Fig. 5 Petri net greedy repairs. Red parts denote repairs. Markings in the reachability graph are indicated as multisets. We indicate visible transition with label l with t_l , i.e. $\lambda^{-1}(l) = t_l$ if $l \neq \tau$

new arcs that connect nodes in V to each other. In contrast, the greedy repair is more efficient, with a total computational cost of $\mathcal{O}(|E| + |V|)$. This cost is derived from $\mathcal{O}(|E| + |V|)$ for locating the two nodes in the reachability graph via the BFS algorithm and $\mathcal{O}(1)$ for adding the new arc.

Note that Theorem 1 is still valid for the greedy approach, because we apply a subset of the changes of the basic approach, and each change guarantees soundness and safety, if taken in isolation.

Repair via log-model alignments

This section proposes an alternative technique to repair process models by associating the activity labels of the discriminating features to the correspondent transitions to be repaired.

This section supports that the initial Petri net $N = (P, T, F, \lambda, m^I, m^F)$, with $\lambda : T \rightarrow \mathcal{A} \cup \{\tau\}$, may contain multiple visible transitions for same label, i.e. $\forall t_1, t_2 \in T, \lambda(t_1) = l \wedge \lambda(t_2) = l \wedge l \neq \tau \not\Rightarrow t_1 = t_2$.

The general idea is to use alignments to map the discriminating features $a >_N b$, identified in Step 3, to valid markings that can be used for repairing the Petri net model as described in the previous sections.

Let us suppose that the relation $a >_N b$ needs to be incorporated into the model, and net contains transitions for each label a and b , i.e. we are in the category (i) of repair as described in “The basic approach” section. The technique to repair the model by leveraging on log-model is here applied as follows:

1. We create an event log $\mathcal{L}_{a>b}^r$ by filtering out the traces that do not contain an event for activity a followed by one for activity b .
2. For each trace $\sigma \in \mathcal{L}_{a>b}^r$, we build the following:
 - (a) We compute the extended alignment $\gamma = \langle (s_L^1, s_M^1, m_1), \dots, (s_L^n, s_M^n, m_n) \rangle$ of σ and N .
 - (b) For each synchronous move $(s_L^i, s_M^i, m_i) \in \gamma$ such that $\lambda(s_M^i) = a$, we add marking m_i to a marking multiset M^a .
 - (c) For each synchronous move $(s_L^j, s_M^j, m_j) \in \gamma$ such that $\lambda(s_M^j) = b$ and $j > 1$, we add the marking m_{j-1} , i.e. reached before firing s_M^j , to a marking multiset M^b .

When every trace in $\mathcal{L}_{a>b}^r$ is considered, we consider the most frequent markings in the multiset M^a and m_b in M^b , denoted as m_a and m_b in M^b , respectively. These markings are ultimately used to repair the Petri net model, as described in “Greedy repair” section.

For category (ii), namely at least one transition for activity a is present in N , but no transition exists for activity b , a similar approach is used, focusing only on the most frequent marking m_a in M^a . This marking can then be used for repairing the Petri net as shown in Fig. 3c. Category (iii), namely a transition for activity b is present in N , but no transition exists for activity a , follows an analogous reasoning as for category (ii).

Example 6 Given an event log $\mathcal{L} = \{\langle a, c, c \rangle^{10}, \langle a, b, c \rangle^{10}, \langle a, b, c, c \rangle^{10}, \langle a, c \rangle^2\}$ and a Petri net N shown in black in Fig. 6. Suppose we want to incorporate the relation $a >_N c$ into N . Filtering the event log to retain only the traces σ s.t. $a >_\sigma c$ yields $\mathcal{L}_{a>c} = \{\langle a, c, c \rangle^{10}, \langle a, c \rangle^2\}$. The optimal alignment of $\langle a, c, c \rangle$ is as follows:

Log moves	a	$\gg$	c	c
Model moves	t_a^1	t_b	t_c^1	t_c^2
Model markings	$[p_2]$	$[p_3]$	$[p_4]$	$[p_6]$

Similarly, the optimal alignment for $\langle a, c \rangle$ is:

Log moves	a	$\gg$	c
Model moves	t_a^2	t_b	t_c^2
Model markings	$[p_5]$	$[p_4]$	$[p_6]$

The two most frequent markings m_a and m_c are $[p_2]$ and $[p_3]$, respectively. Finally, the correspondent repair to the Petri net is highlighted in red in Fig. 6.

Note that selecting specific markings for repairs prevents the inclusion of multiple arcs in the reachability graph, leading to simpler Petri net models similarly to the greedy approach. However, using alignments may increase computational costs (cf. “[Experimental results: computational-time comparison with the state of the art](#)” section).

Theorem 1 remains valid also in this case, following a reasoning similarly to what expressed in “[Greedy repair](#)” section for the greedy approach.

Evaluation

In this section, we present the evaluation of our framework, showing our approach is capable of improving the overall accuracy of process models.

“[Case studies](#)” section presents the case studies and the models used for the experiments, while “[Experimental setup](#)” section describes the experimental setup. In “[Experimental results: fitness & precision](#)” section, we discuss the results in terms of fitness, precision and their balance, comparing them with the state of the art (Fahland and van der Aalst 2015). “[Experimental results: simplicity](#)” section provides a comparison of

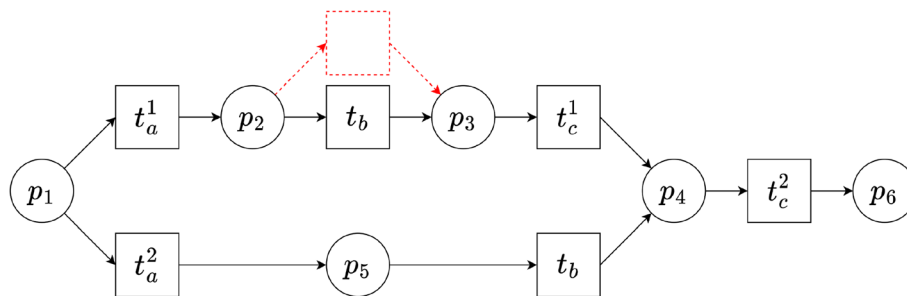


Fig. 6 Petri net log-model alignment repair. Red and dashed parts denote repairs. We indicate visible transition with label / with t_i

Table 3 Event Logs description: number of traces; number of events; average, minimum and maximum number of events per trace

Event log	Traces	Events	Avg ev. per trace	Min ev. per trace	Max ev. per trace
P2P	608	9119	15	2	44
BPIC12W	8616	59302	7	1	74
BPIC17W	30276	240854	8	1	65
RTF	150370	561470	4	2	20
Sepsis	1050	15214	14	3	185

model simplicity between our approach and Fahland and van der Aalst (2015). Additionally, “[Experimental results: simulations quality](#)” section shows the method’s effectiveness in producing models that generate more accurate simulations compared to those created using the initial process model. In “[Experimental results: computational-time comparison with the state of the art](#)” section we compared the efficiency of the different approaches and the state of the art. Finally, in “[Experimental results: noise robustness and scalability](#)” section we analyzed the scalability of the framework with input of increasing complexity.

Case studies

To assess the accuracy and effectiveness of our framework, we evaluated it on five different case studies, each representing different business processes, with an event log provided for each case:

- Purchase to Pay (P2P): a synthetic event log describing a purchasing example process⁶.
- BPIC12W: the subprocess for the workflow-relevant activities, i.e. those starting with W, in the BPI Challenge 2012 event data, a log of a loan application process from a Dutch financial institution⁷.
- BPIC17W: the same subprocess as BPIC12W but referring to the 2016 process executions, which are recorded in the BPI Challenge 2017 event data⁸.
- Road Traffic Fines (RTF): process of managing road traffic fines by a local police force in Italy⁹.
- Sepsis: a real life event log obtained from an Enterprise Resource Planning (ERP) system of a regional hospital in The Netherlands¹⁰.

Table 3 shows a comprehensive description of each event log for each case study.

⁶ <https://fluxicon.com/academic/material>

⁷ <https://doi.org/10.4121/uuid:3926db30-f712-4394-aebc-75976070e91f>

⁸ <https://doi.org/10.4121/uuid:5f3067df-f10b-45da-b98b-86ae4c7a310b>

⁹ <https://doi.org/10.4121/uuid:270fd440-1057-4fb9-89a9-b699b47990f5>

¹⁰ <https://doi.org/10.4121/uuid:915d2bfb-7e84-49ad-a286-dc35f063a460>

The original Petri net models, i.e. before repairs, for the first three case studies were generated by converting the BPMN models publicly available in the repository of the reproducibility package of Camargo et al. (2021), which were discovered through Split Miner (Augusto et al. 2017). For all case studies, we also employed the Inductive Miner (Leemans et al. 2013) to mine additional models. To gain insights into the framework's impact on the initial fitness and precision of the models, we experimented with four different noise thresholds (0.25, 0.5, 0.75, 1). It's worth noting that we could obtain the same models using varying noise thresholds. We also performed experiments with a noise threshold equal to 0: with fitness already equal to 1, our techniques did not alter the model, and neither did the technique by Fahland and van der Aalst (2015). In other words, when the models had fitness equal to 1, all repair techniques returned the original models without any change.

To evaluate the effectiveness of the log-model alignments approach (cf. “Repair via log-model alignments” section) in repairing models with multiple transitions sharing the same label, we also included the normative models derived from Mannhardt (2018) for the Road Traffic Fines and Sepsis case studies. However, these models exhibit an high level of fitness, thus capturing all the observed behaviors. We then modified the normative models by removing the least frequent 10% of transitions, along with their ingoing and outgoing arcs, ensuring that the resulting models remain sound and retain transitions with same labels.

Experimental setup

The framework has been implemented in Python: it takes in input an event log in XES format, and a Petri net in Petri Net Markup Language (PNML)¹¹.

The event log is temporally split into train, validation, and test sets: with respectively 60%, 20% and 20% of the total traces. The training and the validation logs are used by the framework as described in “Framework” section, while the test log is employed for evaluation. We also implemented a Stochastic Petri net simulator in Python for the simulation phase (cf. “Step 1: process simulation” section)¹². In Step 2, CatBoost (Prokhorenkova et al. 2018), a ML method based on gradient boosting over decision trees, is used as discriminator model. SHAP values are computed using the Python's implementation¹³. The validation set/log were also used to perform hyper-parameter optimization during each framework iteration to avoid overfitting and/or underfitting.

During the experiments, we monitored the accuracies in the training and validation sets of the discriminator model during each framework iteration in order to avoid overfitting and/or underfitting. In general, we found optimal results using depth equal to 3, a learning rate of 0.01, and training for 150 iterations.

Experimental results: fitness & precision

In this section, the quality of the obtained models is measured as harmonic means of fitness and precision (van der Aalst et al. 2012). The harmonic means aims to quantify the

¹¹ <https://github.com/franvinci/MLProcessModelRepair>

¹² https://github.com/franvinci/SPN_Simulator

¹³ <https://shap.readthedocs.io>

Table 4 Experimental results using the models obtained via (Fahland and van der Aalst 2015) (SA), and through our basic (RB), greedy (RG) and log-model alignments approach (RA). Results are compared through fitness, precision and their harmonic mean. Column O illustrates the values for the original models, namely before repairing, which were either mined through process-discovery techniques or present in literature. The best harmonic mean for each model is highlighted in bold

Case Study	Input Model	Fitness					Precision					H. Mean									
		O		SA		RB		RG		RA		O		SA		RB		RG		RA	
P2P	Ref.	0.64	1.00	0.95	0.95	0.95	0.95	0.95	0.99	0.13	0.82	0.85	0.82	0.77	0.23	0.88	0.89	0.88	0.89	0.88	
	IM(0.25)	0.63	1.00	1.00	1.00	1.00	1.00	1.00	1.00	0.13	0.81	0.81	0.81	0.77	0.23	0.89	0.89	0.89	0.89	0.89	
	IM(0.5,0.75,1)	0.63	1.00	1.00	1.00	1.00	1.00	1.00	1.00	0.13	0.81	0.81	0.81	0.78	0.23	0.89	0.89	0.89	0.89	0.89	
BPIC12W	Ref.	0.68	1.00	1.00	1.00	1.00	1.00	1.00	0.91	0.38	0.49	0.49	0.49	0.78	0.55	0.65	0.65	0.65	0.65	0.65	
	IM(0.25)	0.84	1.00	1.00	1.00	1.00	1.00	1.00	0.91	0.42	0.65	0.65	0.65	0.87	0.60	0.79	0.79	0.79	0.79	0.79	
	IM(0.5,0.75)	0.62	1.00	0.91	0.91	0.91	0.91	0.91	0.65	0.36	0.49	0.49	0.49	0.63	0.53	0.64	0.64	0.64	0.64	0.64	
BPIC17W	IM(1)	0.61	1.00	1.00	1.00	1.00	1.00	1.00	0.55	0.36	0.36	0.36	0.36	0.58	0.53	0.53	0.53	0.53	0.53	0.53	
	Ref.	0.73	1.00	1.00	1.00	1.00	1.00	1.00	1.00	0.54	0.73	0.73	0.73	0.84	0.70	0.84	0.84	0.84	0.84	0.84	
	IM(0.25)	0.85	1.00	1.00	0.98	0.98	0.98	0.98	0.74	0.46	0.70	0.79	0.77	0.79	0.63	0.82	0.87	0.87	0.87	0.87	
RTF	IM(0.5)	0.57	1.00	0.91	0.79	0.79	0.79	0.79	0.71	0.42	0.61	0.69	0.69	0.63	0.59	0.73	0.79	0.79	0.79	0.81	
	IM(0.75)	0.58	1.00	0.78	0.67	0.67	0.67	0.67	0.63	0.42	0.57	0.67	0.63	0.61	0.59	0.66	0.67	0.67	0.68	0.68	
	IM(1)	0.46	1.00	0.79	0.78	0.78	0.78	0.78	0.36	0.42	0.50	0.55	0.56	0.40	0.59	0.61	0.65	0.65	0.64	0.64	
Sepsis	N	1.00	1.00	-	-	-	-	-	0.81	0.56	-	-	0.81	0.90	0.72	-	-	-	0.90	0.90	
	N*	0.95	0.99	-	-	-	-	-	0.86	0.35	-	-	0.87	0.90	0.51	-	-	-	0.92	0.92	
	IM(0.25)	0.96	1.00	0.99	0.99	0.99	0.99	0.99	0.52	0.44	0.54	0.55	0.57	0.68	0.61	0.70	0.70	0.72	0.72	0.72	
Avg.	IM(0.5)	0.89	1.00	1.00	0.98	0.98	0.98	0.96	0.67	0.45	0.45	0.70	0.68	0.76	0.62	0.62	0.81	0.81	0.80	0.80	
	IM(0.75)	0.93	1.00	0.98	0.98	0.98	0.98	0.99	0.76	0.51	0.78	0.78	0.71	0.84	0.68	0.87	0.87	0.83	0.83	0.83	
	IM(1)	0.61	1.00	0.99	0.99	0.99	0.99	0.64	0.75	0.43	0.78	0.75	0.86	0.67	0.60	0.87	0.86	0.74	0.74	0.74	
Avg.	N	0.99	1.00	-	-	-	-	-	0.26	0.25	-	-	0.26	0.41	0.40	-	-	-	0.41	0.41	
	N*	0.89	0.96	-	-	-	-	-	0.59	0.21	-	-	0.55	0.71	0.34	-	-	-	0.69	0.69	
	IM(0.25)	0.87	1.00	0.93	0.91	0.91	0.91	0.91	0.49	0.20	0.49	0.49	0.49	0.63	0.33	0.64	0.64	0.64	0.64	0.64	
Avg.	IM(0.5)	0.58	1.00	0.95	0.76	0.76	0.76	0.77	0.78	0.20	0.30	0.73	0.64	0.67	0.34	0.45	0.74	0.70	0.70	0.70	
	IM(0.75,1)	0.52	1.00	0.83	0.55	0.55	0.55	0.62	1.00	0.82	0.44	0.78	0.75	0.63	0.34	0.57	0.65	0.68	0.68	0.68	
	Avg.	0.69	1.00	0.95	0.91	0.91	0.91	0.92	0.76	0.38	0.60	0.67	0.65	0.70	0.50	0.72	0.76	0.75	0.75	0.75	

improvement in fitness and/or precision, and their balance. Harmonic means is preferable over arithmetic means because the former yields more penalization than the latter when fitness or precision is significantly lower than the other (i.e., balancing less). Separately, we also report the simplicity of the models as introduced in Blum (2015). Results have finally been also compared with the approach by Fahland and van der Aalst (2015).

The results are reported in Table 4, where fitness, precision and their harmonic mean are reported for the original model (label O), namely before the repair, our framework (columns with gray background) using the basic approach (label RB), using greedy repairs (label RG) and using alignments repair (label RA), and the best result by the work of Fahland and van der Aalst (2015) (label SA). Each row indicates a different model. $IM(0.25), \dots, IM(1)$ denote the original models discovered via Inductive Miner with noise threshold $0.25, \dots, 1$. Ref. refers to the original “reference” models that are available in the reproducibility package of Camargo et al. (2021), i.e. obtained by using Split Miner, N indicates the normative models from Mannhardt (2018), while N* the modified models obtained as described in “Case studies” section. Note that results for the models from Mannhardt (2018) using basic and greedy approaches are not present since they contain transitions with same labels (cf. “Repair via log-model alignments” section).

The reported results are computed on the test event log, not used for repair. All our approaches revealed themselves to be effective for almost all models employed in our experiments: compared with 19 out of the original 23 models (83%), they provide models that improve the harmonic means of fitness and precision, while consistently outperforming the method presented by Fahland and van der Aalst (2015) (except for one case, where the harmonic means is the same). In fact, the improvement with respect to Fahland and van der Aalst (2015) is unsurprising, since the method in Fahland and van der Aalst (2015) maximizes fitness by incorporating all the behaviour of the event log into the Petri net model, leading to models with perfect fitness but low precision, whereas our method only adds the most discriminating behaviour without penalizing precision, yielding to a balanced process model.

Considering the four original models where we do not improve on the harmonic means, they are already characterized by a high means, which does not allow further improvement. In fact, in an attempt to improve, we worsen the quality of those models: this clearly does not invalidate our technique, because one can easily check whether or not the repaired is actually a better model, and can discard the changes when they are counterproductive.

Comparing our approaches between them, we observe that the greedy produces repaired models with higher or similar harmonic means of fitness and precision. Moreover, the log-model alignments approach produces performances similar to the greedy one and it shows its effectiveness in his usage also in models that present transitions with same labels.

Experimental results: simplicity

We also compared the level of simplicity of the original and repaired models. Table 5 reports the results. Clearly, since our approaches incorporate additional behaviour, the repaired models are inevitably less simple. However, comparing the simplicity of the models generated by Fahland and van der Aalst (2015) and by our approaches,

we observe that, on average, our greedy and log-model alignments approaches produce simpler repaired models: the average simplicity is 0.55 for the approach by Fahland et al. while it is 0.58 for our greedy approach and 0.57 for the alignments one. A visual inspection of the models repaired via our greedy approach shows that the models are clearly readable (see Fig. 7).

In general, it follows that *our greedy and log-model alignments approaches are able to produce simpler models with a higher harmonics means of fitness and precision, if compared with* (Fahland and van der Aalst 2015).

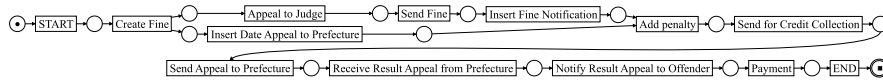
Experimental results: simulations quality

In this section we show the ability of the proposed framework in repairing the input process models for obtaining more accurate simulated event logs. In fact, the framework is built to find inaccuracies between a simulated event log using the starting model and the real event log. The inaccuracies are then used for repairing the model to obtain a simulated event log which presents less deviations from the real one.

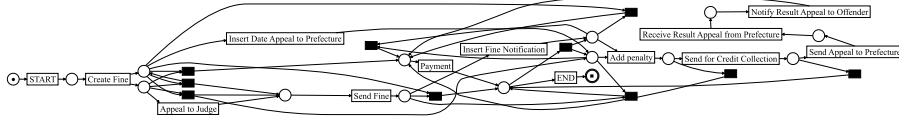
To assess simulation performances, we generated simulated event logs from the process model, as described in “[Step 1: process simulation](#)” section, and compared them

Table 5 Simplicity results using the models obtained via (Fahland and van der Aalst 2015) (SA), and through our basic (RB), greedy (RG) and log-model alignments approach (RA). Column O illustrates the values for the original models, namely before repairing, which were either mined through process-discovery techniques or present in literature

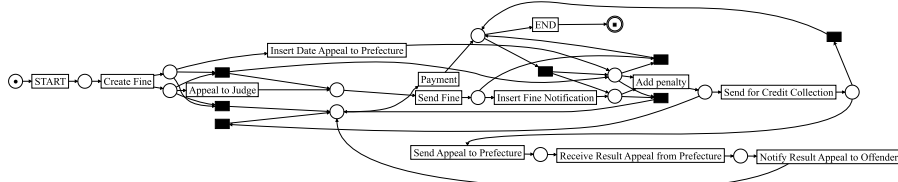
Case Study	Input Model	Simplicity				
		O	SA	RB	RG	RA
P2P	Ref.	0.92	0.63	0.78	0.81	0.78
	IM(0.25)	0.96	0.64	0.75	0.75	0.75
	IM(0.5,0.75,1)	1.00	0.64	0.75	0.75	0.75
BPIC12W	Ref.	0.82	0.55	0.60	0.62	0.60
	IM(0.25)	0.71	0.59	0.61	0.61	0.61
	IM(0.5,0.75)	0.78	0.53	0.60	0.60	0.60
	IM(1)	0.68	0.48	0.55	0.55	0.53
BPIC17W	Ref.	0.79	0.59	0.64	0.64	0.64
	IM(0.25)	0.68	0.55	0.48	0.56	0.56
	IM(0.5)	0.73	0.51	0.35	0.44	0.44
	IM(0.75)	0.67	0.51	0.32	0.47	0.47
	IM(1)	0.71	0.49	0.29	0.42	0.42
RTF	N	0.63	0.56	-	-	0.61
	N*	0.82	0.53	-	-	0.73
	IM(0.25)	0.70	0.55	0.36	0.58	0.58
	IM(0.5)	0.73	0.57	0.17	0.52	0.52
	IM(0.75)	0.77	0.54	0.59	0.66	0.58
	IM(1)	1.00	0.52	0.49	0.57	0.83
Sepsis	N	0.62	0.56	-	-	0.54
	N*	0.80	0.53	-	-	0.56
	IM(0.25)	0.67	0.56	0.05	0.56	0.35
	IM(0.5)	0.75	0.51	0.11	0.33	0.32
	IM(0.75,1)	0.77	0.50	0.11	0.54	0.36
Avg.		0.78	0.55	0.45	0.58	0.57



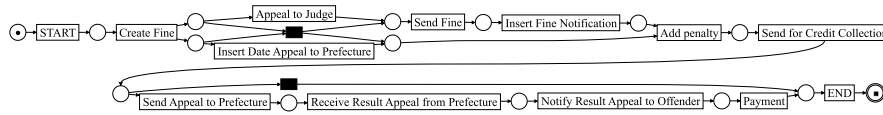
(a) RTF IM(1) original Petri net. Simplicity: 1.00. Fitness-Precision Harmonic Mean: 0.67.



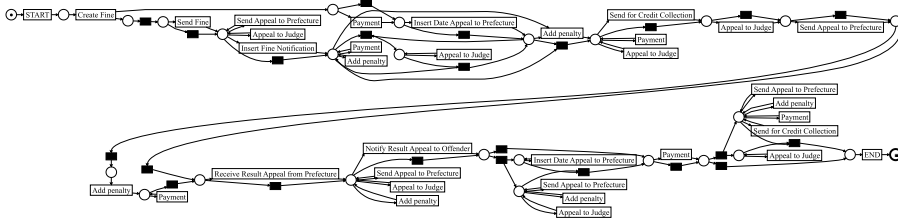
(b) RTF IM(1) repaired Petri net using the basic approach. Simplicity: 0.49. Fitness-Precision Harmonic Mean: 0.87.



(c) RTF IM(1) repaired Petri net using the greedy approach. Simplicity: 0.57. Fitness-Precision Harmonic Mean: 0.86.



(d) RTF IM(1) repaired Petri net using the log-model alignment approach. Simplicity: 0.83. Fitness-Precision Harmonic Mean: 0.74.



(e) RTF IM(1) repaired Petri net using [Fahland and van der Aalst \(2015\)](#). Simplicity: 0.52. Fitness-Precision Harmonic Mean: 0.60.

Fig. 7 Road Traffic Fine (RTF) Petri net mined using Inductive Miner with threshold equal to 1 (IM(1)) and the Petri nets obtained via repairs

to the actual event log using the control-flow measurements proposed in Chapela-Campa et al. (2025): Control-flow Log Distance (CFLD) and N-Gram Distance (NGD).

In our evaluation, we compared the results obtained from the original models, with those from the *best* repaired process model. Here, we refer to the *best* process model as the one that achieves the highest harmonic mean of fitness and precision across the repaired models generated using our approaches. To mitigate the inherent stochasticity of simulations, we generated 10 event logs for each process model. Then, using a *t-test* with a 90% confidence level, we computed confidence intervals for these results. Particularly, we found that 10 simulations were sufficient as they produced a standard deviation of the results close to zero. For the N-Gram Distance we set N equal to 3 and 4, finding that larger values did not produce additional changes. Table 6 presents

Table 6 Simulation results using the original models (O) and our best results from our framework (*bR*)

Case Study	Input Model	CFLD		3GD		4GD	
		O	<i>bR</i>	O	<i>bR</i>	O	<i>bR</i>
P2P	Ref.	0.425(±0.004)	0.156(±0.009)	0.316(±0.004)	0.074(±0.009)	0.346(±0.005)	0.096(±0.010)
	IM(0.25)	0.421(±0.002)	0.156(±0.021)	0.291(±0.004)	0.092(±0.015)	0.316(±0.006)	0.116(±0.015)
	IM(0.5,0.75,1)	0.431(±0.003)	0.120(±0.026)	0.334(±0.004)	0.090(±0.016)	0.370(±0.004)	0.116(±0.016)
BPIC12W	Ref.	0.427(±0.002)	0.274(±0.003)	0.543(±0.002)	0.433(±0.004)	0.584(±0.002)	0.485(±0.004)
	IM(0.25)	0.280(±0.003)	0.136(±0.004)	0.308(±0.004)	0.107(±0.004)	0.373(±0.003)	0.141(±0.003)
	IM(0.5,0.75)	0.505(±0.001)	0.334(±0.003)	0.602(±0.001)	0.538(±0.004)	0.642(±0.001)	0.604(±0.005)
	IM(1)	0.515(±0.001)	0.359(±0.002)	0.598(±0.001)	0.605(±0.003)	0.639(±0.001)	0.671(±0.003)
BPIC17W	Ref.	0.343(±0.001)	0.154(±0.002)	0.473(±0.001)	0.261(±0.003)	0.5595(±0.001)	0.315(±0.003)
	IM(0.25)	0.267(±0.003)	0.211(±0.002)	0.340(±0.002)	0.281(±0.002)	0.388(±0.002)	0.319(±0.002)
	IM(0.5)	0.481(±0.001)	0.219(±0.003)	0.482(±0.001)	0.290(±0.002)	0.559(±0.001)	0.361(±0.002)
	IM(0.75)	0.491(±0.001)	0.439(±0.002)	0.532(±0.001)	0.494(±0.003)	0.575(±0.001)	0.529(±0.003)
	IM(1)	0.631(±0.001)	0.407(±0.003)	0.670(±0.001)	0.463(±0.002)	0.706(±0.001)	0.513(±0.002)
RTF	N	0.102(±0.002)	0.103(±0.003)	0.171(±0.003)	0.172(±0.002)	0.213(±0.004)	0.214(±0.002)
	N*	0.104(±0.001)	0.094(±0.002)	0.182(±0.002)	0.172(±0.003)	0.223(±0.002)	0.212(±0.003)
	IM(0.25)	0.163(±0.002)	0.162(±0.002)	0.239(±0.003)	0.234(±0.002)	0.303(±0.003)	0.296(±0.003)
	IM(0.5)	0.207(±0.001)	0.186(±0.002)	0.198(±0.003)	0.202(±0.002)	0.236(±0.004)	0.237(±0.003)
	IM(0.75)	0.045(±0.001)	0.046(±0.001)	0.084(±0.001)	0.082(±0.001)	0.097(±0.001)	0.093(±0.002)
	IM(1)	0.545(±0.001)	0.056(±0.002)	0.609(±0.001)	0.058(±0.003)	0.685(±0.001)	0.081(±0.003)
Sepsis	N	0.443(±0.005)	0.447(±0.005)	0.529(±0.007)	0.533(±0.008)	0.635(±0.007)	0.640(±0.007)
	N*	0.472(±0.006)	0.469(±0.006)	0.562(±0.007)	0.547(±0.010)	0.676(±0.006)	0.659(±0.008)
	IM(0.25)	0.565(±0.008)	0.578(±0.008)	0.653(±0.010)	0.609(±0.009)	0.742(±0.009)	0.687(±0.005)
	IM(0.5)	0.533(±0.004)	0.554(±0.004)	0.749(±0.005)	0.721(±0.004)	0.818(±0.004)	0.816(±0.004)
	IM(0.75,1)	0.580(±0.005)	0.512(±0.004)	0.781(±0.002)	0.720(±0.004)	0.844(±0.002)	0.784(±0.003)
Avg.		0.390	0.267	0.445	0.338	0.501	0.391

the results, where columns O and *bR* indicate the original and the *best* repaired process model respectively, with the best-performing values highlighted in bold. Specifically, we conclude that the repaired process model outperforms the original if its confidence interval is entirely above that of the original model. If the intervals overlap, we cannot draw a definitive conclusion, while a confidence interval entirely below the original indicates poorer performance than the initial model.

The results prove that our framework generally improves simulation performances. For the CFLD we achieved a reduction in distance in 16 out of 23 cases, with only one case showing a worst performance. Overall, we achieved a general improvement of 31.54% compared to the original models. For the 3GD and 4GD metrics we obtained similar results, with average improvements of 24.04% and 21.96%, respectively. Moreover, the 3GD and 4GD measures highlight the framework capability to improve the handling of long-term dependencies.

Experimental results: computational-time comparison with the state of the art

We monitored the computational times using the three approaches, running the experiments on the same machine equipped with an Intel Core i5-8300H CPU @ 2.30GHz and 16GB RAM. Table 7 summarizes key statistics of computational times across the experiments. Our findings indicate that the method by Fahland and van der Aalst (2015) is in

Table 7 Computational times (in seconds) per each case study and correspondent models, using the approach proposed by Fahland and van der Aalst (2015) - identified by SA - versus our basic approach (RB), our greedy approach (RG) and the approach based on log-model alignments (RA)

Case Study	Input Model	SA	RB	RG	RA
P2P	Ref.	4	16	11	20
	IM(0.25)	6	18	15	32
	IM(0.5,0.75,1)	6	21	16	27
BPIC12W	Ref.	13	147	146	202
	IM(0.25)	13	73	68	130
	IM(0.5,0.75)	14	240	236	283
	IM(1)	17	167	151	175
BPIC17W	Ref.	4226	350	344	573
	IM(0.25)	67	409	304	558
	IM(0.5)	728	447	282	559
	IM(0.75)	71	455	450	537
	IM(1)	74	402	398	732
RTF	N	20	-	-	69
	N*	320	-	-	60
	IM(0.25)	23	142	135	191
	IM(0.5)	58	3920	180	259
	IM(0.75)	14	138	131	165
	IM(1)	53	172	171	189
Sepsis	N	157	-	-	587
	N*	386	-	-	79
	IM(0.25)	94	6781	235	686
	IM(0.5)	44	2824	275	1041
	IM(0.75,1)	32	1296	95	111
Avg.		280	948	192	316
Std.		857	1701	124	275
Median		44	240	171	191

general faster than ours, but there are some cases where it exhibits significantly higher computational times. This is due to the generation of complex intermediate models that make it hard to compute model-to-trace alignments, thereby increasing the computational times. Moreover, we observed that the greedy method is the fastest among those we proposed. In fact, the basic approach introduces multiple arcs into the reachability graph (cf. “[Step 4: process model repair](#)” section), increasing the complexity of intermediate models. Also, the log-model alignments method requires the computation of alignments, which is computationally intensive.

Experimental results: noise robustness and scalability

This section elaborates on two further aspects that are relevant for the practical applicability of the framework: robustness to noise, and scalability. Noise robustness refers to the capability of our repair techniques to distinguish genuine deviations and noise. Genuine deviations are those that tend to repeat with a certain degree of frequency, which is clearly different from noise, which is conversely characterized by random deviations that do not repeat with a consistent pattern. Scalability conversely focuses on the capability of our techniques to repair models of large size.

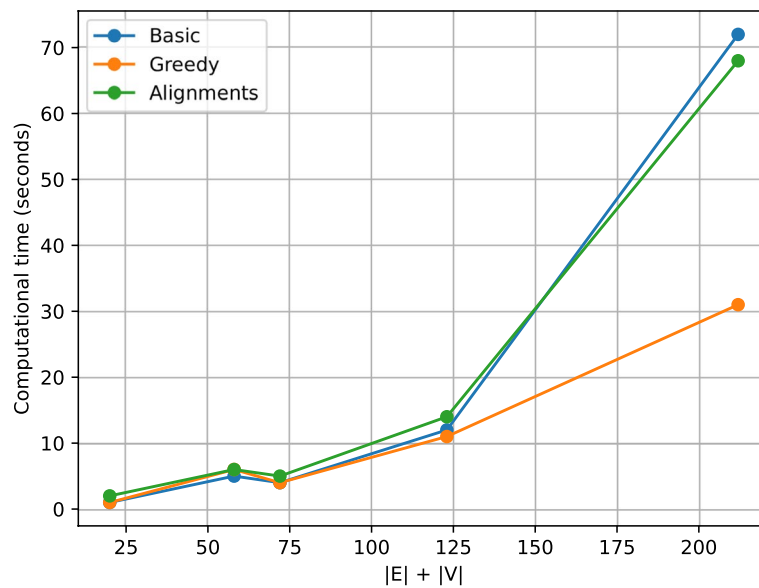


Fig. 8 Computation times (in seconds) in relation to process model complexity for each proposed approach. Model complexity is quantified by the total number of edges E and nodes V of the process model reachability graph

These two problems have been investigated by generating five Petri nets of larger and larger sizes and accordant event logs with 500 traces with and without noise employing the PLG2 tool by Burattin (2016).

For the scalability analysis, we then randomly swapped two consecutive transitions in the Petri nets. We conducted experiments on these altered process models, monitoring the computational time for each of our repair approaches. Figure 8 reports the computation time (y axis) for Petri nets of increasing size (x axis). A model's size is here measured as the sum of the number of arcs and nodes of its reachability graph. Indeed, in “[Greedy repair](#)” section, the theoretical complexity of the repair is correlated to the size of the reachability graph, which is certainly correlated to the number of places and transitions of the Petri nets but better characterizes the complexity of the model from an algorithmical viewpoint (cf. a Petri net with ten transitions in parallel versus another with the same ten transition in a sequence). The results show that the computational times of the greedy method grow linearly with model complexity, while the basic approach would follow a nearly-quadratic trend. These observation align with the computational cost analysis reported in “[Greedy repair](#)” section. Regarding the alignment-based model-repair approach, it seems to practically follow a trend similar to the basic approach, although it is based on computing alignments, which is at least an NP-hard problem.

To assess the noise tolerance, we injected random noise levels of 10%, 20%, and 30% to the logs by randomly altering the order of some events in the control flow, aiming to assess whether any noise level caused unwanted model repairs. We used the five Petri nets that PLG directly generated, and we didn't those in which we swapped transitions. When attempting to repair the Petri nets with these three noisy event logs, our framework did not make any change to the model. Recall that the noisy-free event logs were fully compliant with the model, and hence no changes are expected to occur after applying our repair technique. This showed that the noise was identified as such, and ignored.

This is related to the ability of the Machine-Learning classifier to detect the noise and not overfit the data. Consequently, SHAP does not identify any significant features influencing the outputs (i.e. whether a trace is real or simulated), thereby no repairs are applied to the original process model. In our experiments, we employed CatBoost, but the management of noise and outliers is a feature present in the vast majority of classifiers.

Related works

The work by Fahland and van der Aalst (2015) is among the first ones that address the model-repair problem to enhance fitness, but is not the only one. Polyvyanyy et al. (2016) puts forward methods to define process model repair as an optimization problem, where costs are assigned to various repair actions. However, this work aims to maximizing the fitness, as authors also clearly state, relegating precision as second-class citizen.

Mitsyuk et al. (2017) propose a model-repair approach able to achieve a high level of fitness, but, as authors themselves admit, at a cost of dropping precision. Other works address the model repair and try to improve the model precision (Kalenkova et al. 2020; Rennert and van der Aalst 2024).

In a recent work, Genga et al. (2024) address the challenge of balancing fitness and precision by proposing a repair methodology based on anomalous local instance graphs. However, their approach assumes the presence of a user that chooses among a set of anomalous instances the one to be used for repairing. We then decided to compare our framework with the work of Fahland and van der Aalst (2015) since it is the one, with an available implementation, proposing the problem of repairing process models similarly as our approach.

Other existing literature proposes interactive frameworks for process model repair, in which possible differences between the model and the log are displayed to the user who manually repairs the process model (Armas Cervantes et al. 2017; Pourbafrani and van der Aalst 2022). These research works do not aim at an automated model repair, which serves a different purpose: repairing the model, using feedback from humans.

Repair a process model is also linked to *concept drift* (cf. Revoredo (2023)): one wants to repair the model to align it with the behaviour after, i.e. when the process changes over time and then the model does not align the reality anymore. Revoredo (2023) framed process model repair as a theory revision problem. In this work, the event log and the process model are partitioned to repair only changeable parts of the model that are manually selected by practitioners, considering only traces representing acceptable behaviour. In case of concept drift one can select the part of the model subject of the process changes, and repair that.

Incremental process discovery (see, e.g., Schuster et al. (2023)) is also related to model repair, but the goal is different as the former does not aim to obtain models that are as close as possible to a reference original process model.

Meneghello et al. (2022) proposed a framework that uses decision-tree models to discriminate the traces observed in the event log from those allowed by a simulation model. The resulting decision trees should be used as input to subsequently change the simulation model in multiple iterations, so that ultimately the traces of the event log and those obtained from simulation cannot be distinguished, namely the resulting decision tree has very low accuracy. However, the changes need to be manually implemented

by process analysts, a step that is sometimes difficult, because the decision tree gives ambiguous explanations.

Note that repairing a model can be regarded as ensuring soundness of the model (see, e.g., Gambini et al. (2011); Lohmann and Fahland (2014)), which is a relevant, yet orthogonal problem.

Towards multi-perspective process model repair

An avenue of future work is to not limit the repair to the control-flow perspective, but to extend it to consider the other perspectives, as well. This can be achieved by including features in the discriminator that refer to other perspectives:

Resource Perspective. For each transition T , a resource categorical feature can be incorporated, which takes on a value equal to the name of the resource that performed A . If transition T is performed multiple times, one needs to approximate it as, e.g., the last resource performing A . Adding a feature for each pair of transition and resource would likely generate too many features, with a high risk of leading to overfitting. If a discrimination is found on the feature for, e.g., T , the set of resources allowed to perform T is changed. Namely, if resource R performs T , then the trace is real or, resp., from the simulated log, then R is added or, resp., removed from the set of resources allowed to perform T .

Time Perspective. For each transition T , a numerical feature can be added to indicate the execution duration of instances of T (or, average if executed multiple times). If a discrimination is found on the feature for a specific transition, the modelling of the distribution of T 's duration is considered incorrect, which thus needs to be (re-)discovered from data, as, e.g., indicated in Camargo et al. (2020). Currently, directly-follow features in Definition 8 are boolean-typed, to which true and false values are assigned depending on whether or not the sequence of two transitions is observed in a trace (cf. Eq. 2), respectively. The boolean value can potentially be replaced by the time elapsed between two subsequent transitions, thus being able to discriminate on the basis of the transition's waiting times. However, waiting time might be related to exogenous conditions and events that are outside the "universe" of the process model, or alternatively to the unavailability of resources. The latter case is clearly related to the resource perspective, more than to the time. It is thus challenging to (semi)automatically repair the model to accommodate the correct waiting times. In a spirit of separation of concerns, one could firstly fix the resource perspective: this would mean that the resulting waiting time is related to exogenous conditions, annotating the arc with waiting-time characterization (simple averages, distributions, etc.).

Data Perspective. The relevant aspects related to the data perspective of a process model are largely focused on the data-driven decisions that determine how process instances are executed. These decisions are typically modeled via guards attached to transitions (cf. de Leoni (2022)). For each transition T with a guard G , a feature can be included that indicate whether G evaluated true when T was performed. Similarly to the time perspective, if a discrimination is found of a guard-

related feature, that is an indication that the guard was likely wrong and would be required to be rediscovered.

Conclusion

A process model is one of the key ingredients for several techniques for the analysis, improvement and enactment of processes. A process model is also employed to engage with process' stakeholders into discussion on how the process is being executed, possibly in comparison with how it ought be carried out.

For any of the purposes that we just mentioned, the process model needs to reasonably summarize all the legitimate behavior (high fitness), and only that (high precision). Unfortunately, process models, either hand-made or mined from event logs, are often-times characterized by not scoring well in both fitness and/or precision. In fact, obtaining a model with high fitness is sometimes contrasting with having one with precision.

This article introduced a model-repair framework that aims to improve an initial process model so as to maximize the harmonic means of fitness and precision. In fact, a higher harmonic means guarantees that their balance is better. This is a novelty with respect to the largest body of research on model repair (cf. "[Related works](#)" section), which conversely tends to give more importance to fitness (typically), or to precision (seldom).

In summary, our framework idea is as follows. The framework uses two input elements: a process model, represented as a Petri net, and an event log. The process model is initially used to generate a simulated event log, which is then compared with a real event log to pinpoint differences. The differences are unveiled by training a ML discriminator model. Since an accurate process model would generate logs that are indistinguishable from the original event log, the ML model should perform poorly. If it is conversely able to discriminate, explainable AI techniques, specifically SHAP, are used to explain and pinpoint the discriminatory model features. These features are now actionable to allow for an automated model repair: we illustrate how to use them to this aim. Three approaches are being proposed for the automated model repair, two of which were previously introduced in Vinci and de Leoni (2024): one basic and complete, one more greedy in deciding how to repair. However, both the approaches are actionable when the process model does not contain multiple visible transitions of the same label. This assumption holds when models are mined via the major process discovery algorithms, such as Inductive Miner (Leemans et al. 2013) or Split Miner (Augusto et al. 2017). However, several hand-made models do not match this assumptions, such as two out of the five used in evaluation.

The evaluation was based on five processes and several models, hand-made taken from the literature or mined through Inductive Miner. The evaluation shows that our repair technique can produce better models - in terms of harmonic means of fitness and precision - than what alternative, state-of-the-art techniques would produce, which are also better than the original models.

This article is an extension of our previous work by Vinci and de Leoni (2024), and provides the following additional contributions:

1. An alternative approach is proposed, which is based on log-model alignments and can also repair process models with multiple visible transition of the same label (cf. “[Repair via log-model alignments](#)” section).
2. We performed experiments on four new models, for two case studies, from Mannhardt (2018) and their derivatives (cf. “[Case studies](#)” section), which our framework is only able to repair thanks to the introduction of the new repair approach based on log-model alignments.
3. Not only does the evaluation show successful application of our technique to produce better repaired models, but also does it show the benefits in real settings when the goal is to use process models as starting point to create a simulation: the simulation models created on the basis of our repair process models are more realistic than those obtained from the original process models (cf. “[Experimental results: simulations quality](#)” section).
4. On the presentation side, the article is enriched (i) through numerous examples to better describe the framework, and the proposed evaluation approaches, and also (ii) includes some figures of original and repaired process models, which allow readers to appreciate how our framework can produce simpler models.
5. Analyses and experiments were conducted to successfully assess the robustness of the repair technique to noise and the scalability to models of larger size. The theoretical and test-based analyses of complexity are indeed reported in “[Greedy repair](#)” and “[Experimental results: noise robustness and scalability](#)” sections, respectively.

A definitive assessment of the accuracy of simulation models (cf. item 3 in the list above) has to consider the simulation stochasticity: one single simulation might not be representative. To tackle this aspect, we performed several simulation runs and conducted t-tests for assessing the representativeness of the results obtained (cf. “[Experimental results: simulations quality](#)” section).

This work assumes that the simulated event logs are nearly complete with respect to the process model (Yang et al. 2012). We acknowledge that this assumption can be a threat of validity, and we aim in the future to evaluate the completeness of the simulated event log. Currently, we only repair models by allowing addition behavior, while it might potentially be the case in which a better balancing between fitness and precision can be achieved by restricting behavior: so do we aim to extend the technique so as to restrict behavior.

Authors' contributions

All the authors contributed to the formulation of the problem and to the design of a suitable solution framework and an experimental setting. F. Vinci implemented the framework, and conducted the experiments. All the authors contributed to the analysis of the results, and to the writing and revision of the manuscript.

Funding

The research work by F. Vinci is financially supported by MUR (PNRR) and University of Padua. The work by M. de Leoni has been partly supported by the European Union - Next Generation EU under the National Recovery and Resilience Plan (NRRP), Mission 4 Component 2 Investment 1.1 - Call PRIN 2022 PNRR No. 1409 of September 14, 2022 of Italian Ministry of University and Research; Project P20222XM58 (subject area: SH - Social Sciences and Humanities) *Emergency medicine 4.0: an integrated data-driven approach to improve emergency department performances*.

Data availability

No datasets were generated or analysed during the current study.

Declarations

Competing interests

The authors declare no competing interests.

Received: 14 November 2024 Accepted: 27 February 2025

Published online: 03 April 2025

References

- Armas Cervantes A, van Beest NRTP, La Rosa M, Dumas M, García-Bañuelos L (2017) Interactive and incremental business process model repair. In: Panetto H, Debruyne C, Gaaloul W, Papazoglou M, Paschke A, Ardagna CA, Meersman R (eds) *On the Move to Meaningful Internet Systems. OTM 2017 Conferences*, Springer International Publishing, Cham, pp 53–74. https://doi.org/10.1007/978-3-319-69462-7_5
- Augusto A, Conforti R, Dumas M, La Rosa M (2017) Split miner: discovering accurate and simple business process models from event logs. In: 2017 IEEE International Conference on Data Mining (ICDM), pp 1–10. <https://doi.org/10.1109/ICDM.2017.9>
- Blum FR (2015) Metrics in process discovery. In: Technical Report TR/DCC-2015-6, Computer Science Department, University of Chile
- Burattin A (2016) PLG2: multiperspective process randomization with online and offline simulations. In: Azevedo L, Cabanillas C (eds) *Proceedings of the BPM Demo Track 2016 Co-located with the 14th International Conference on Business Process Management (BPM 2016)*, Rio de Janeiro, Brazil, September 21, 2016. *CEUR Workshop Proceedings*, vol 1789. CEUR-WS.org, pp 1–6
- Burke A, Leemans SJJ, Wynn MT (2021) Stochastic process discovery by weight estimation. In: Leemans S, Leopold H (eds) *Process Mining Workshops*, Springer International Publishing, Cham, pp 260–272. https://doi.org/10.1007/978-3-030-72693-5_20
- Camargo M, Dumas M, González-Rojas O (2020) Automated discovery of business process simulation models from event logs. *Decis Support Syst* 134:113284. <https://doi.org/10.1016/j.dss.2020.113284>
- Camargo M, Dumas M, González-Rojas O (2021) Discovering generative models from event logs: data-driven simulation vs deep learning. *PeerJ Comput Sci*. <https://doi.org/10.7717/peerj-cs.577>
- Chapela-Campa D, Benchekroun I, Baron O, Dumas M, Krass D, Senderovich A (2025) A framework for measuring the quality of business process simulation models. *Inf Syst* 127:102447. <https://doi.org/10.1016/j.is.2024.102447>
- de Leoni M (2022) *Foundations of process enhancement*, Springer International Publishing, Cham, pp 243–273. https://doi.org/10.1007/978-3-031-08848-3_8
- Fahland D, van der Aalst WM (2015) Model repair – aligning process models to reality. *Inf Syst* 47:220–243. <https://doi.org/10.1016/j.is.2013.12.007>
- Gambini M, La Rosa M, Migliorini S, ter Hofstede AHM (2011) Automated error correction of business process models. In: Rinderle-Ma S, Toumani F, Wolf K (eds) *Business Process Management*, Springer Berlin Heidelberg, Berlin, Heidelberg, pp 148–165. https://doi.org/10.1007/978-3-642-23059-2_14
- Genga L, Rossi F, Diamantini C, Storti E, Potena D (2024) Model repair supported by frequent anomalous local instance graphs. *Inf Syst* 122:102349. <https://doi.org/10.1016/j.is.2024.102349>
- Kalenkova A, Carmona J, Polyvyanyy A, La Rosa M (2020) Automated repair of process models using non-local constraints. In: Janicki R, Sidorova N, Chatain T (eds) *Application and Theory of Petri Nets and Concurrency*, Springer International Publishing, Cham, pp 280–300. https://doi.org/10.1007/978-3-030-51831-8_14
- Leemans SJJ, Fahland D, van der Aalst WMP (2013) Discovering block-structured process models from event logs - a constructive approach. In: Colom JM, Desel J (eds) *Application and Theory of Petri Nets and Concurrency*, Springer Berlin Heidelberg, Berlin, Heidelberg, pp 311–329. https://doi.org/10.1007/978-3-642-38697-8_17
- Lohmann N, Fahland D (2014) Where Did I Go Wrong? In: Sadiq S, Soffer P, Völzer H (eds) *Business Process Management*, Springer International Publishing, Cham, pp 283–300. https://doi.org/10.1007/978-3-319-10172-9_18
- Lundberg SM, Lee SI (2017) A unified approach to interpreting model predictions. In: *Proceedings of the 31st International Conference on Neural Information Processing Systems*, Curran Associates Inc., Red Hook, NY, USA, NIPS'17, pp 4768–4777. <https://doi.org/10.5555/3295222.3295230>
- Mannhardt F (2018) Multi-perspective process mining. Eindhoven: Technische Universiteit Eindhoven, 379 p. (SIKS dissertation series). <https://research.tue.nl/en/publications/multi-perspective-process-mining>
- Meneghello F, Fracca C, de Leoni M, Asnicar F, Turco A (2022) A framework to improve the accuracy of process simulation models. In: Guizzardi R, Ralyté J, Franch X (eds) *Research Challenges in Information Science*, Springer International Publishing, Cham, pp 142–158. https://doi.org/10.1007/978-3-031-05760-1_9
- Mitsyuk A, Lomazova I, Shugurov I, van der Aalst W (2017) Process model repair by detecting unfitting fragments? In: *Supplementary Proceedings of the Sixth International Conference on Analysis of Images, Social Networks and Texts (AIST 2017)*. *CEUR Workshop Proceedings*. CEUR-WS, pp 301–313
- Polyvyanyy A, van der Aalst WMP, ter Hofstede AHM, Wynn MT (2016) Impact-driven process model repair 25(4). <https://doi.org/10.1145/2980764>
- Pourbafrani M, van der Aalst WMP (2022) Interactive process improvement using simulation of enriched process trees. In: Hacid H, Aldwairi M, Bouadjenek MR, Petrocchi M, Faci N, Outay F, Beheshti A, Thamsen L, Dong H (eds) *Service-Oriented Computing – ICSOC 2021 Workshops*, Springer International Publishing, Cham, pp 61–76. https://doi.org/10.1007/978-3-031-14135-5_5
- Prokhorenkova L, Gusev G, Vorobev A, Dorogush AV, Gulin A (2018) Catboost: unbiased boosting with categorical features. In: Bengio S, Wallach H, Larochelle H, Grauman K, Cesa-Bianchi N, Garnett R (eds) *Advances in Neural Information Processing Systems*, Curran Associates, Inc., vol 31. <https://doi.org/10.48550/arXiv.1810.11363>

- Rennert C, van der Aalst WMP (2024) Improving precision in process trees using subprocess tree logs. In: De Smedt J, Soffer P (eds) *Process Mining Workshops*, Springer Nature Switzerland, Cham, pp 110–122. https://doi.org/10.1007/978-3-031-56107-8_9
- Revoredo K (2023) On the use of domain knowledge for process model repair. *Softw Syst Model* 22:1099–1111. <https://doi.org/10.1007/s10270-022-01067-0>
- Schuster D, Föcking N, van Zelst SJ, van der Aalst WMP (2023) Incremental discovery of process models using trace fragments. In: Di Francescomarino C, Burattin A, Janiesch C, Sadiq S (eds) *Business Process Management*, Springer Nature Switzerland, Cham, pp 55–73. https://doi.org/10.1007/978-3-031-41620-0_4
- van der Aalst WMP, Adriansyah A, van Dongen B (2012) Replaying history on process models for conformance checking and performance analysis. *WIREs Data Min Knowl Discov*. <https://doi.org/10.1002/widm.1045>
- van der Aalst WMP (1998) The application of petri nets to workflow management. *J Circ Syst Comput* 08(01):21–66. <https://doi.org/10.1142/S0218126698000043>
- van der Aalst WMP (2015) *Business Process Simulation Survival Guide*, Springer Berlin Heidelberg, Berlin, Heidelberg, pp 337–370. https://doi.org/10.1007/978-3-642-45100-3_15
- Vinci F, de Leoni M (2024) Repairing process models through simulation and explainable AI. In: Marrella A, Resinas M, Jans M, Rosemann M (eds) *Business Process Management*, Springer Nature Switzerland, Cham, pp 129–145. https://doi.org/10.1007/978-3-031-70396-6_8
- Yang H, van Dongen B, ter Hofstede A, Wynn M, Wang J (2012) Estimating completeness of event logs. *BPM Center Report BPM-12-04*, BPMcenter.org, <http://www.workflowpatterns.com/bpmreports/2012/BPM-12-04.pdf>

Publisher's Note

Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.